GAS Data Warehouse

Complete Table Documentation

Schemas covered: etl • stg • dim • fact • ml (128 tables)

Azure Cosmos DB for PostgreSQL (Citus) / Azure Data Factory

Table of Contents

(Right-click and choose "Update Field" in Word to populate this list.)

1. Overview

This document catalogs every table in the gas_db data warehouse across all five schemas — etl, stg, dim, fact and ml — with each table's DDL, constraints and relationships, indexes, and how it is populated. It was compiled directly from the warehouse DDL export, the fact/dim/ml stored-procedure source, the index definition script, and the Azure Data Factory (ADF) ARM template for the factory, rather than from memory — every population narrative traces back to a specific stored procedure, Copy Activity, or mapping data flow found in those files.

gas_db runs on Azure Cosmos DB for PostgreSQL (Citus), a distributed PostgreSQL cluster. Two consequences of that show up repeatedly in the tables below. First, Citus does not support foreign-key constraints across distributed tables the way a single-node PostgreSQL instance would, and most primary-key/unique constraints in the original DDL export are commented out (wrapped in /* ... */) rather than active. Where the source SQL still documents an intended primary key, unique constraint, or foreign key, this document calls it out as a 'logical' constraint — the relationship the schema was designed around — while noting explicitly that it is not enforced by the database engine. Second, the warehouse uses application-level surrogate-key sequences (dim.*_seq) and PL/pgSQL upsert patterns (INSERT ... ON CONFLICT, NOT EXISTS anti-joins, DISTINCT ON dedup) to maintain data integrity in place of engine-enforced constraints.

1.1 Medallion Architecture

The warehouse follows a medallion architecture: source systems (Cosmos DB order/event documents, and the GMS, NGE, GSH and GXS platform databases) land in ADLS Gen2 as Bronze (hourly Hive-partitioned NDJSON/Parquet), get distributed into per-entity Silver staging tables in the stg schema, and are finally merged into the Gold layer's dim, fact and ml schemas by PL/pgSQL stored procedures. The etl schema holds a single control table, etl.bronze_partition_registry, that tracks which hourly Bronze partitions have been processed.

Two large 'fan-out' procedures sit at the center of the orders/events pipeline: fact.usp_distribute_silver_transaction_entities() reads every order document landed in stg.silver_all_transaction_entities for the current Bronze partition and, filtering by document type, NOT EXISTS-inserts the right rows into ten downstream per-entity staging tables (silver_transaction_header, silver_transaction_item, silver_transaction_payment, silver_transaction_combo_items, silver_item_modifiers, silver_upsell_recommendations, silver_modifier_recommendations, silver_modifier_impressions, silver_modifier_interactions, silver_transaction_refunds). fact.usp_distribute_silver_gem_events() does the same job for the events side, fanning stg.silver_all_gem_events out into stg.silver_kiosk_events and stg.silver_cep_incidents.

From there, each per-entity staging table has one or more dedicated fact.usp_silver_*_to_fact() / fact.usp_gem_*_to_fact() procedures that read new rows past a checkpoint stored in fact.watermarktable, deduplicate with DISTINCT ON, and upsert into the corresponding Gold fact table (usually via INSERT ... ON CONFLICT DO UPDATE, sometimes via a plain NOT EXISTS-guarded insert). A parallel set of dim.usp_refresh_*() procedures follows the same shape for dimension tables, reading from stg.dim_* staging snapshots (which are themselves simple full-copy Copy Activities from GMS/Cosmos DB sources, not Bronze/Silver-distributed) and upserting into dim.*.

The ml schema sits one layer further downstream: its eight tables are rebuilt daily (mostly full TRUNCATE/DELETE-and-rebuild rather than incremental) from the dim and fact Gold tables, producing week- or session-grained feature tables consumed by the Data Science team's models — including the fuzzy item/modifier name-matching table ml.modifier_item_match, which reuses the token-sort-ratio logic ported from Python's rapidfuzz.

1.2 Pipeline Orchestration & Load Order

The ADF factory contains 67 pipelines, but only four are actually attached to a schedule trigger:

- GASOrders (daily) -> DimensionsFactsMaster — the main orchestrator. It runs, in order: the Dimensions step (DimensionUpdateOrdersToAnalyticsDb, which itself calls DimensionMaster's four sub-pipelines: DimensionMenuItems, DimensionFrequentCustomer, DimensionMenuModifiers, DimensionKiosks, plus CopyGOUViewsToGAS), then TruncateStagingTables (etl.truncate_silver_staging(), clearing the Silver staging tables on a 10-minute sliding watermark), then BronzeSilverGASMaster (running BronzeEventsToSilverAndGAS and BronzeOrdersToSilverAndGAS in parallel — the Bronze-to-Silver-to-Gold fan-out described above), then GEM_GSH_GOU_Master, and finally Facts (FactsMaster: gasTransactionsToAnalyticsDb -> UpsellRecommendationsToGAS -> ModifierUpsellRecommendationsToGAS -> AbortedOrdersAndItemsToPG -> OccasionSurveyOrdersFeedback, in that order).
- GEMEvents (daily) -> GEM_GSH_GOU_Master directly — the same pipeline that also runs as a step inside DimensionsFactsMaster, suggesting some overlap/redundancy between the two trigger paths that may be worth reviewing with the team. Internally it chains: CopyGOUViewsToGAS (preceded by the dimView/dimElement lookup steps) -> gemEventsToAnalyticsDb (factDeviceEvent -> factCEPIncidents -> factUserBehaviour) -> GEMUserBehaviourEventsToPG -> gshDeviceHealthToAnalyticsDbFactDeviceState -> gshDeviceTelemetryToAnalyticsDb -> gemOrderTimingtoAnalyticsDb -> gemUserCheckedInToPG -> DimensionABTests.
- RefreshMLTables (daily) -> RefreshMLTables — rebuilds the ml schema in a fixed dependency order: WeatherML -> MenuEntitiesML -> ModifierEntitiesML -> TransactionsML -> UpsellAnalysisML -> ModifierInteractionsML -> ModifierImpressionsML -> MasterKeysForDuplicateItems -> GetLocationAndItemStatistics.
- TwiceDaily -> GrubbrrInstallBaseDashboard — refreshes the install-base staging tables and dim.kioskdetails, then rebuilds dim.vw_grubbrrinstallbase and feeds the customer/location menu-preference fact tables.

A number of other pipelines exist in the factory but are not wired to any of the four triggers or called by any other pipeline — for example BronzeEventsToGAS/BronzeOrdersToGAS (superseded by the ...AndSilver... variants), DimensionMasterItemsAndModifiers (superseded by the split DimensionMenuItems/DimensionMenuModifiers pipelines), the various Test* pipelines, and the ML training-data extraction pipelines (TrainingDataForML, CompressedTrainingDataForML, SampleDataForMLTraining, etc.). These are noted individually where they are a table's only known population path, and flagged as manual/legacy/on-demand where relevant.

1.3 How to Read Each Entry

Each table entry below follows the same structure:

- DDL — the CREATE TABLE statement as defined in the warehouse, reformatted for readability.
- Constraints & Relationships — any primary key, unique constraint, or foreign key found in the source (most are commented-out / logical-only under Citus, as explained above), plus a plain-language description of how the table relates to others.
- Indexes — every CREATE INDEX statement targeting the table.
- How it's populated — the specific stored procedure(s), ADF Copy Activity, or mapping data flow that write to the table, its source tables/systems, and the incremental-load or dedup logic involved (watermark checkpoints, DISTINCT ON, NOT EXISTS anti-joins, ON CONFLICT upserts).

Tables with no identified population source in the reviewed artifacts are called out explicitly rather than guessed at, with a best-effort explanation (static reference data, a manual backup snapshot, or a retired/superseded object).

2. ETL Schema - Pipeline Control Tables

One control table used to checkpoint Bronze ADLS partition processing for the orders and events pipelines.

2.1 etl.bronze_partition_registry

Watermark/checkpoint ledger that tracks, for every hourly Bronze ADLS partition (orders and events), whether it has been picked up and processed by the Bronze-to-Silver-to-GAS pipelines. This is the mechanism that lets ADF pipelines process each hourly folder exactly once and resume safely after a failure.

DDL

```
CREATE TABLE etl.bronze_partition_registry (
    dateid integer NOT NULL,
    layer text,
    entity text NOT NULL,
    partition_path text,
    partition_date date,
    partition_year smallint,
    partition_month smallint,
    partition_day smallint,
    partition_hour smallint,
    status text DEFAULT 'pending'::text,
    file_count integer,
    started_at timestamp without time zone,
    completed_at timestamp without time zone,
    adf_pipeline_run_id text,
    error_message text,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	bronze_partition_registry_pkey	entity, dateid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Rows are seeded ahead of time by a one-off/periodic maintenance script (etl_bronze_partition_registry.sql) that cross-joins dim.datedim against the entity list ('orders', 'events') to pre-generate one pending row per entity per hour, with partition_path built as a Hive-style path (e.g. orders/raw/YYYY/MM/DD/HH).

During normal operation, the BronzeOrdersToSilverAndGAS and BronzeEventsToSilverAndGAS pipelines (and their BronzeOrdersToGAS/BronzeEventsToGAS predecessors) query this table with status IN ('pending') and dateid within a safe processing window (current time minus a 1-hour buffer for late-arriving files) to decide which partition to pick up next, typically one partition at a time ordered by dateid ascending.

After a partition's Copy Activity completes, the pipeline updates the row's status to 'completed' (or 'not found' if the folder was empty), stamping started_at, completed_at, file_count and adf_pipeline_run_id ('@{pipeline().RunId}'). This UPDATE is issued directly from ADF Lookup/Script activities, not from a dedicated stored procedure.

This table has no upstream table dependency; it is a self-contained ETL control table and sits at dependency level 0.

3. STG Schema - Bronze/Silver Staging Tables

Forty-five staging tables: raw Bronze landing tables, per-entity Silver tables produced by the two distribution procedures, and full-snapshot copies of dimension reference data from GMS/NGE/Cosmos DB sources.

3.1 stg.dim_catalog

Landing copy of the GMS product catalog header (catalog id/name, organization, master/standalone/ECM flags) used to refresh dim.catalog.

DDL

```
CREATE TABLE stg.dim_catalog (
    catalogid character varying(50) NOT NULL,
    catalogname character varying(255),
    organizationid character varying(40),
    is_catalog_deleted boolean,
    catalog_created_on timestamp without time zone,
    catalog_modified_on timestamp without time zone,
    gem_company_id character varying(255),
    gem_location_id character varying(255),
    is_sync_in_progress boolean,
    is_standalone boolean,
    is_master boolean,
    is_ecm_enabled boolean,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`catalog_pkey`	`catalogid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for the GMS public.item_master catalog-level attributes, populated by a Copy Activity in the DimensionOrganizationLocationCatalog pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_catalog (DISTINCT ON catalogid, NOT EXISTS-guarded insert of new catalogs plus an UPDATE of changed attributes), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the

rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.2 stg.dim_category_hierarchy

Landing copy of the location-level category-to-menu-item hierarchy mapping used to refresh dim.category_hierarchy.

DDL

```
CREATE TABLE stg.dim_category_hierarchy (
    organizationid text,
    locationid text NOT NULL,
    mapping_created_on timestamp without time zone,
    mapping_modified_on timestamp without time zone,
    is_mapping_active boolean,
    is_mapping_deleted boolean,
    catalogid text,
    catalogname text,
    catalog_created_on timestamp without time zone,
    catalog_modified_on timestamp without time zone,
    is_catalog_active boolean,
    is_catalog_deleted boolean,
    categoryid text NOT NULL,
    categoryname text,
    category_created_on timestamp without time zone,
    category_modified_on timestamp without time zone,
    is_category_active boolean,
    is_category_deleted boolean,
    menuitemid text,
    entitytype text,
    item_class_type integer,
    menuitemname text,
    item_created_on timestamp without time zone,
    item_modified_on timestamp without time zone,
    is_item_active boolean,
    is_item_deleted boolean,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	`location_category_menuitem_unq`	`locationid, categoryid, menuitemid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for GMS public.item_master category/menu-item hierarchy records, populated by a Copy Activity in the DimensionMasterItemsAndModifiers / DimensionMenuItems pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_category_hierarchy (DISTINCT ON locationid, categoryid, menuitemid; NOT EXISTS-guarded insert with an UPDATE for is_mapping_active/mapping_modified_on changes), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.3 stg.dim_cep_subscriptions

Landing copy of which locations/organizations are subscribed to which GEM Connector/CEP event categories.

DDL

```
CREATE TABLE stg.dim_cep_subscriptions (
    id text,
    cep_subscriptions text,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'stgCEPSubscriptions' Copy Activity in RefreshInstallBaseDataset, sourced from the gemSubscription Cosmos DB container.

Read (joined, not INSERT-targeted) by dim.usp_refresh_organization to help populate organization-level CEP subscription flags in dim.organization.

3.4 stg.dim_frequentcustomer

Landing copy of loyalty/frequent-customer profile records (name, email, phone) used to refresh dim.frequentcustomer.

DDL

```
CREATE TABLE stg.dim_frequentcustomer (
    frequentcustomerid text NOT NULL,
    firstname text,
    lastname text,
    email text,
    phone text,
    source text,
    organizationid text,
    createddate text,
    lastorderdate text,
    ordercount integer DEFAULT 0 NOT NULL,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	frequent_customer_pk	frequentcustomerid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for NGE loyalty program customer records (ngeFrequentCustomer Cosmos container), populated by a Copy Activity in the DimensionFrequentCustomer pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_frequentcustomer (DISTINCT ON frequentcustomerid, NOT EXISTS-guarded insert with an UPDATE of changed contact attributes), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.5 stg.dim_itemcategory

Landing copy of menu category definitions (name, active/deleted flags, owning catalog) used to refresh dim.itemcategory.

DDL

```
CREATE TABLE stg.dim_itemcategory (
    locationid text NOT NULL,
    categoryid text NOT NULL,
    categoryname text NOT NULL,
    catalogid text,
    is_category_active boolean,
    is_category_deleted boolean,
    category_created_on timestamp without time zone,
    category_modified_on timestamp without time zone,
    is_alcoholic boolean,
    number_of_items smallint,
    number_of_sub_categories smallint,
    number_of_item_variations smallint,
    number_of_combos smallint,
    number_of_combo_families smallint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for GMS public.item_master category records, populated by a Copy Activity in the DimensionMasterItemsAndModifiers / DimensionMenuItems pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_itemcategory (DISTINCT ON locationid, categoryid, NOT EXISTS-guarded insert with an UPDATE for name/active-flag changes), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.6 stg.dim_kiosk

Landing copy of kiosk device attributes (name, app version, test-kiosk flag, device type) sourced from the device-health platform, used to refresh dim.kiosk.

DDL

```
CREATE TABLE stg.dim_kiosk (
    locationid text NOT NULL,
    kioskid text NOT NULL,
    kioskname text,
    appversion text,
    istestkiosk boolean,
    devicetype character varying(50),
    devicecreatedon timestamp without time zone,
    devicedeletedon timestamp without time zone,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for gsh.device (GSH device-health platform), populated by a Copy Activity in the DimensionKiosks pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_kiosk (DISTINCT ON locationid, kioskid, NOT EXISTS-guarded insert with an UPDATE of changed attributes), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.7 stg.dim_kiosk_appearance

Landing copy of kiosk UI/appearance/branding configuration, one of six inputs consolidated into dim.kioskdetails for the install-base dashboard.

DDL

```
CREATE TABLE stg.dim_kiosk_appearance (
    locationid text,
    kiosk_receipt_settings text,
    kiosk_fonts text,
    kiosk_appearance_text_overrides text,
    kiosk_appearance_style_options text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by 'stgDimKioskAppearance' in RefreshInstallBaseDataset, sourced from the gasCosmosLocation container.

Consumed (DISTINCT ON locationid) by dim.usp_refresh_dim_location_kiosk_details / dim.usp_refresh_location_kiosk_details, which merge it with the other five stg.dim_* install-base tables into dim.kioskdetails via an ON CONFLICT (locationid) DO UPDATE.

3.8 stg.dim_kiosk_config

Landing copy of kiosk operational configuration (the widest of the install-base staging tables, ~30 columns) — feeds dim.kioskdetails.

DDL

```
CREATE TABLE stg.dim_kiosk_config (
    locationid text,
    scanners text,
    item_special_request text,
    legal_copy_enabled boolean,
    ada_configuration text,
    calculate_default_modifier_price boolean,
    track_kiosk_user_behavior boolean,
    loyalty_feature boolean,
    pickup_flow boolean,
    pos_auto_applied_discount boolean,
    search_functionality_enabled boolean,
    recent_orders_enabled boolean,
    play_card_config text,
    round_up_for_charity boolean,
    calories_enabled boolean,
    scan_and_go_enabled boolean,
    age_verification text,
    tips_settings text,
    business_hours_config text,
    order_types text,
    localization text,
```

```
    loyalty_display_settings text,
    preorder_popup_enabled boolean,
    preorder_popup_text text,
    disclaimer_text text,
    order_limit_config text,
    menu_behavior_config text,
    perform_pos_status_check boolean,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by 'stgDimKioskConfig' in RefreshInstallBaseDataset, sourced from the gasCosmosLocation container.

Consumed by dim.usp_refresh_dim_location_kiosk_details / dim.usp_refresh_location_kiosk_details alongside the other install-base staging tables, merged into dim.kioskdetails via ON CONFLICT (locationid) DO UPDATE.

3.9 stg.dim_location_kiosks

Landing copy of the location-to-kiosk association list — the anchor table for the install-base merge (one row per location, joined against the other five stg.dim_* config tables).

DDL

```
CREATE TABLE stg.dim_location_kiosks (
    id text,
    locationid text,
    companyid text,
    devicetype text,
    syncversion text,
    kiosks text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by 'stgDimLocationKiosks' in RefreshInstallBaseDataset, sourced from the gasCosmosLocation container.

Read as the base FROM table by dim.usp_refresh_dim_location_kiosk_details / dim.usp_refresh_location_kiosk_details, LEFT JOINed to stg.dim_pos_provider, stg.dim_loyalty_configuration, stg.dim_payment_provider, stg.dim_kiosk_config and stg.dim_kiosk_appearance to build the consolidated dim.kioskdetails row.

3.10 stg.dim_loyalty_configuration

Landing copy of which loyalty program is configured per location — one of the six install-base inputs merged into dim.kioskdetails.

DDL

```
CREATE TABLE stg.dim_loyalty_configuration (
    locationid text,
    loyalty_provider text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for NGE loyalty configuration records (ngeLoyaltyConfigurations Cosmos container), populated by a Copy Activity in the RefreshInstallBaseDataset pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_dim_location_kiosk_details / dim.usp_refresh_location_kiosk_details (DISTINCT ON locationid, loyalty_provider, merged via LEFT JOIN into dim.kioskdetails), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.11 stg.dim_menuitem

Landing copy of menu item master attributes (name, entity type, calories, protein, etc.) used to refresh dim.menuitem.

DDL

```
CREATE TABLE stg.dim_menuitem (
    menuitemid text NOT NULL,
    menuitemname text NOT NULL,
    entitytype text,
    calories text,
    protein numeric,
    sugar numeric,
    fat numeric,
```

```
    is_alcoholic boolean,
    is_vegetarian_item boolean,
    is_vegan_item boolean,
    has_allergen boolean,
    item_class_type integer,
    is_active boolean,
    is_deleted boolean,
    gms_created_on timestamp without time zone,
    gms_modified_on timestamp without time zone,
    itemunitprice numeric(12,3),
    price_changed_on timestamp without time zone,
    catalogid text,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for GMS public.item_master menu-item records, populated by a Copy Activity in the DimensionMasterItemsAndModifiers / DimensionMenuItems pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_menuitem (DISTINCT ON menuitemid, NOT EXISTS-guarded insert with an UPDATE for nutritional/attribute changes), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.12 stg.dim_modifier

Landing copy of modifier master attributes (catalog, name, min/max quantity) used to refresh dim.modifier.

DDL

```
CREATE TABLE stg.dim_modifier (
    modifierid character varying(50) NOT NULL,
    catalogid character varying(50),
    modifiername character varying(255),
    min_quantity integer,
    max_quantity integer,
    allow_quantity_increment boolean,
    increment_step integer,
    calories text NOT NULL,
    calories_text text,
    is_modifier_active boolean NOT NULL,
    is_modifier_deleted boolean NOT NULL,
    modifier_created_on timestamp without time zone,
    modifier_modified_on timestamp without time zone,
    is_modifier_default boolean,
    modifier_default_quantity integer,
```

```
    is_invisible boolean,
    classification integer,
    price numeric(12,3),
    price_changed_on timestamp without time zone,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for GMS public.item_master modifier records, populated by a Copy Activity in the DimensionMasterItemsAndModifiers / DimensionMenuModifiers pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_modifier (DISTINCT ON modifierid, NOT EXISTS-guarded insert with an UPDATE for attribute changes), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.13 stg.dim_modifiergroup

Landing copy of modifier group master attributes (name, catalog, min/max selection) used to refresh both dim.modifier_group and dim.modifier_group_mapping.

DDL

```
CREATE TABLE stg.dim_modifiergroup (
    modifiergroupid character varying(50) NOT NULL,
    modifiergroupname character varying(510) NOT NULL,
    catalogid character varying(50) NOT NULL,
    max_selection integer,
    min_selection integer,
    free_count integer,
    pos_linked_entity_id character varying(50),
    is_active boolean NOT NULL,
    is_deleted boolean NOT NULL,
    created_on timestamp without time zone,
    modified_on timestamp without time zone,
    negative_modifier_behavior integer,
    created_by character varying(255),
    modified_by character varying(255),
    max_aggregate_count integer,
    min_aggregate_count integer,
    increment_step integer,
    slider_mode boolean DEFAULT false NOT NULL,
    slider_mode_modifier boolean DEFAULT false NOT NULL,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	modifier_group_master_pkey	modifiergroupid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for GMS public.item_master modifier-group records, populated by a Copy Activity in the DimensionMasterItemsAndModifiers / DimensionMenuModifiers pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_modifiergroup and dim.usp_refresh_modifiergroup_modifier_mapping (both DISTINCT ON modifiergroupid, NOT EXISTS-guarded insert with attribute UPDATE), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.14 stg.dim_modifiergroup_modifier_mapping

Landing copy of the modifier-to-modifier-group glue mapping used to refresh dim.modifier_group_mapping.

DDL

```
CREATE TABLE stg.dim_modifiergroup_modifier_mapping (
    modifier_mapping_id character varying(50) NOT NULL,
    modifierid character varying(50) NOT NULL,
    modifiergroupid character varying(50) NOT NULL,
    catalogid character varying(50),
    is_mapping_active boolean,
    is_mapping_deleted boolean,
    mapping_created_on timestamp without time zone,
    mapping_modified_on timestamp without time zone,
    is_default boolean,
    default_quantity integer,
    allow_quantity_increment boolean,
    increment_step integer,
    min_quantity integer,
    max_quantity integer,
    calories_text text,
    is_invisible boolean,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	modfrgrp_modfr_unq	modifiergroupid, modifierid
PRIMARY KEY	modifier_group_modifier_glue_pkey	modifier_mapping_id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for GMS public.item_master modifier-group-to-modifier mappings, populated by a Copy Activity in the DimensionMasterItemsAndModifiers / DimensionMenuModifiers pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_modifiergroup_modifier_mapping, which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.15 stg.dim_occasionsurvey

Landing copy of guest-feedback survey definitions (survey/question/selection type) used to refresh dim.occasionsurvey.

DDL

```
CREATE TABLE stg.dim_occasionsurvey (
    organizationid text NOT NULL,
    surveyid text NOT NULL,
    surveyname text,
    surveytype integer,
    question_type integer,
    selection_type integer,
    survey_status integer,
    is_deleted boolean,
    created_on timestamp without time zone,
    modified_on timestamp without time zone,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'dimOccasionSurveyCopy' Copy Activity in OccasionSurveyOrdersFeedback, sourced from the gasSurveyCosmos container.

Consumed (DISTINCT ON organizationid, surveyid) by dim.usp_refresh_occasionsurvey, which performs a NOT EXISTS-guarded insert plus attribute UPDATE into dim.occasionsurvey.

3.16 stg.dim_organization

Landing copy of organization master attributes (33 columns) sourced from the public GOU views — used to refresh dim.organization.

DDL

```
CREATE TABLE stg.dim_organization (
    id character varying(40) NOT NULL,
    name character varying(255) NOT NULL,
    address1 character varying(255),
    address2 character varying(255),
    city character varying(255),
    state character varying(255),
    zipcode character varying(20),
    country character varying(255),
    organizationtype smallint,
    status smallint,
    phonenumber character varying(20),
    email character varying(255),
    isdeleted boolean DEFAULT false,
    createdon timestamp without time zone,
    createdby character varying(255),
    modifiedon timestamp without time zone,
    modifiedby character varying(255),
    active boolean,
    timezone character varying(50),
    coordinates text,
    dayofweek integer,
    hour integer,
    minutes integer,
    roundupforcharity boolean,
    is_ecm_enabled boolean,
    is_cep_enabled boolean,
    is_concessionaire_enabled boolean,
    is_smart_upsells_enabled boolean,
    is_feedback_survey_enabled boolean,
    is_digital_menu_board_enabled boolean,
    is_digital_menu_default_format_enabled boolean,
    cep_subscriptions text,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	organization_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'stgDimOrganization' Copy Activity in CopyGOUViewsToGAS, sourced from the public.organization view.

Consumed by dim.usp_refresh_organization, which LEFT JOINs it to dim.kioskdetails and stg.dim_cep_subscriptions and performs an INSERT ... ON CONFLICT (id) DO UPDATE into dim.organization.

3.17 stg.dim_organizationlocation

Landing copy of the organization-to-location association (which locations belong to which organization) — used to refresh both dim.organizationlocation and dim.userlocation.

DDL

```
CREATE TABLE stg.dim_organizationlocation (
    organizationid character varying(40) NOT NULL,
    organizationname character varying(255),
    locationid character varying(40) NOT NULL,
    locationname character varying(255) NOT NULL,
    organizationtype smallint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	organizationid_locationid_pk	organizationid, locationid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'stgDimOrganizationLocation' Copy Activity in DimensionOrganizationLocationCatalog, sourced from the public.vw_organizationlocation view.

Consumed (DISTINCT ON organizationid, locationid) by dim.usp_refresh_organizationlocation, which joins it to dim.organization and performs a NOT EXISTS-guarded insert plus attribute UPDATE into dim.organizationlocation.

3.18 stg.dim_payment_provider

Landing copy of which payment processor is configured per location — one of the six install-base inputs merged into dim.kioskdetails.

DDL

```
CREATE TABLE stg.dim_payment_provider (
    locationid text,
    payment_provider text,
    syscosmosts bigint,
```

```
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for gasCosmosLocation payment-provider configuration, populated by a Copy Activity in the RefreshInstallBaseDataset pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_dim_location_kiosk_details / dim.usp_refresh_location_kiosk_details (DISTINCT ON locationid, payment_provider, merged via LEFT JOIN into dim.kioskdetails), which upserts (INSERT ... ON CONFLICT / NOT EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.19 stg.dim_pos_provider

Landing copy of which POS connector is configured per location — one of the six install-base inputs merged into dim.kioskdetails.

DDL

```
CREATE TABLE stg.dim_pos_provider (
    locationid text,
    pos_provider text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Landing table for cosmosConnectorConfiguration container, populated by a Copy Activity in the RefreshInstallBaseDataset pipeline — a full snapshot copy (source table is small and re-copied in full on every run rather than incrementally).

Consumed by dim.usp_refresh_dim_location_kiosk_details / dim.usp_refresh_location_kiosk_details (DISTINCT ON locationid, pos_provider, merged via LEFT JOIN into dim.kioskdetails), which upserts (INSERT ... ON CONFLICT / NOT

EXISTS-guarded UPDATE pattern) the rows into the corresponding Gold dim table, then this staging table is truncated/refreshed on the next run rather than being watermark-filtered.

3.20 stg.fact_devicestate

Landing copy of device-health snapshot events from the GSH platform, incrementally loaded — the direct source for fact.devicestate.

DDL

```
CREATE TABLE stg.fact_devicestate (
    id bigint,
    healthdatatype text,
    locationid text,
    companyid text,
    deviceid text,
    devicetype text,
    status text,
    statusmessage text,
    healthdatatime timestamp without time zone,
    statuschangetime timestamp without time zone,
    inserttime timestamp without time zone,
    version text,
    devicedatatime timestamp without time zone,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'gshDeviceHealth' / 'gshDeviceHealthFullLoad' Copy Activities in gshDeviceHealthToAnalyticsDbFactDeviceState, sourced from gsh.devicehealth (with a full-load variant for backfill).

fact.usp_gsh_devicehealth_to_fact_devicestate reads new rows (checkpointed against fact.watermarktable), dedups with DISTINCT ON (locationid, deviceid, healthdatatime), joins dim.kiosk and dim.organization, and inserts into fact.devicestate.

3.21 stg.fact_devicetelemetry

Landing copy of device telemetry (signal/heartbeat) events from the GSH platform — the direct source for fact.devicetelemetry.

DDL

```
CREATE TABLE stg.fact_devicetelemetry (
    deviceid text,
    locationid text,
```

```
    dateid integer,
    cpuvalue numeric(10,5),
    memoryvalue numeric(10,5),
    cputimestamp timestamp without time zone,
    memorytimestamp timestamp without time zone,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'gshDeviceTelemetry' Copy Activity in gshDeviceTelemetryToAnalyticsDb, sourced from gsh.devicetelemetry.

fact.usp_gsh_devicetelemetry_to_fact_devicetelemetry reads new rows (checkpointed against fact.watermarktable), dedups with DISTINCT ON (locationid, deviceid, dateid), joins dim.kiosk and dim.organization, and performs an INSERT ... ON CONFLICT (locationid, deviceid, dateid) DO UPDATE into fact.devicetelemetry.

3.22 stg.fact_itemssurvey

Landing copy of per-item guest-feedback survey responses from the NGE orders-feedback feed — one of two sources for fact.itemssurvey.

DDL

```
CREATE TABLE stg.fact_itemssurvey (
    locationid text,
    dateid integer,
    surveyid text,
    surveytransid text,
    orderid text,
    itemid text,
    itemrating text,
    surveytransstatus text,
    surveyissuedtimestamp text,
    surveycompletedtimestamp text,
    surveylocaltimestamp timestamp without time zone,
    nge_syscosmosts bigint,
    sourceid integer,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'stgFactItemsSurvey' Copy Activity in OccasionSurveyOrdersFeedback, sourced from the ngOrdersFeedback container.

fact.usp_nge_update_itemssurvey reads new rows (checkpointed against fact.watermarktable), dedups with DISTINCT ON (locationid, orderid, itemid, surveyid, surveytransid), joins fact.transactionheader, dim.organizationlocation and dim.occasionsurvey, and inserts into fact.itemssurvey (also touching dim.menuitem for enrichment).

3.23 stg.fact_occasionsurveydetail

Landing copy of order-level (occasion) guest-feedback survey responses from the NGE orders-feedback feed — the direct source for fact.occasionsurveydetail.

DDL

```
CREATE TABLE stg.fact_occasionsurveydetail (
    locationid text,
    dateid integer,
    surveyid text,
    surveytransid text,
    orderid text,
    surveyrating text,
    surveytransstatus text,
    surveyissuedtimestamp text,
    surveycompletedtimestamp text,
    surveylocaltimestamp timestamp without time zone,
    syscosmosts bigint,
    sourceid integer,
    surveytype integer,
    ordersessionid text,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'stgFactOccasionSurveyDetail' Copy Activity in OccasionSurveyOrdersFeedback, sourced from the ngOrdersFeedback container.

fact.usp_stg_occasionsurveydetail_to_fact reads new rows (checkpointed against fact.watermarktable), dedups with DISTINCT ON (locationid, orderid, surveyid, surveytransid), joins fact.transactionheader, dim.organizationlocation, dim.occasionsurvey and stg.silver_kiosk_events, and inserts into fact.occasionsurveydetail.

3.24 stg.gem_failed_order_job_notifications

Landing copy of failed background-job notifications from the GEM platform (e.g. order-sync failures) — the direct source for fact.gem_failed_order_job_notifications.

DDL

```
CREATE TABLE stg.gem_failed_order_job_notifications (
    incidentid TEXT COLLATE pg_catalog."default",
    application TEXT COLLATE pg_catalog."default",
    organizationid TEXT COLLATE pg_catalog."default",
    locationid TEXT COLLATE pg_catalog."default",
    eventmodule TEXT COLLATE pg_catalog."default",
    eventcategory TEXT COLLATE pg_catalog."default",
    eventtype TEXT COLLATE pg_catalog."default",
    eventtoken TEXT COLLATE pg_catalog."default",
    incidentcount INTEGER,
    firstoccurred TEXT COLLATE pg_catalog."default",
    lastoccurred TEXT COLLATE pg_catalog."default",
    incidenttype TEXT COLLATE pg_catalog."default",
    notificationtypeid TEXT COLLATE pg_catalog."default",
    syscosmosts BIGINT,
    sysinserttime TIMESTAMP WITHOUT TIME ZONE
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'stgFailedOrderNotifications' Copy Activity in gemEventsToAnalyticsDb, sourced from the gemJobs container.

fact.usp_stg_gem_failed_order_job_notifications_to_fact reads new rows (checkpointed against fact.watermarktable), dedups with DISTINCT ON (incidentid, eventtoken), and inserts into fact.gem_failed_order_job_notifications; the same rows are also joined by fact.usp_silver_cep_incidents_to_fact_cep_incidents for CEP incident enrichment.

3.25 stg.kioskdetails

A narrower, older-looking staging table for kiosk configuration (7 columns) that predates the six-table install-base staging set (stg.dim_location_kiosks, stg.dim_kiosk_config, etc.).

DDL

```
CREATE TABLE stg.kioskdetails (
    id text,
    locationid text,
    kiosks text,
    devicetype text,
```

```
    syncversion text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed ADF ARM template or PL/pgSQL source references this table as a source or target. It appears to be a superseded precursor to the current install-base staging design (dim.kioskdetails is now built from six separate stg.dim_* tables via dim.usp_refresh_dim_location_kiosk_details).

Recommend confirming with the team whether this table is still required before relying on it; it may be safe to drop as part of any cleanup.

3.26 stg.lookup_silver_transaction_header

A 41-column table shaped like a lookup/reference variant of stg.silver_transaction_header.

DDL

```
CREATE TABLE stg.lookup_silver_transaction_header (
    id integer,
    transactionheaderid text,
    orderid text,
    locationid text,
    kioskid text,
    ordersessionid text,
    dateid integer,
    orderdateutc text,
    orderdatelocal timestamp without time zone,
    orderstatus text,
    ordertype integer,
    numberofitems smallint,
    numberofpayments smallint,
    ordersredeemedrewards numeric(12,3),
    ordersubtotal numeric(12,3),
    ordertotal numeric(12,3),
    ordertax numeric(12,3),
    ordertip numeric(12,3),
    orderdiscount numeric(12,3),
    orderbalance numeric(12,3),
    paymentstatus text,
    sourcefile text,
    createddate timestamp without time zone,
    charityamount numeric(12,3),
    orderservicecharge numeric(12,3),
    businessdate date,
    syscosmosts bigint,
```

```
    channel text,
    guestcount integer,
    frequentcustomerid text,
    customername text,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts references this table. Given the naming and column shape, it most likely was a one-off lookup/staging snapshot created for a specific investigation or migration and left in place, rather than an actively maintained pipeline table.

3.27 stg.modifier_recommendation_sessions

Landing copy of modifier upsell recommendation sessions built directly from Cosmos DB order documents (an ADF mapping-data-flow product, not a Bronze/silver-distribution product like the silver_* tables above).

DDL

```
CREATE TABLE stg.modifier_recommendation_sessions (
    locationid text NOT NULL,
    transactionheaderid text NOT NULL,
    ordersessionid text,
    orderid text,
    modifier_impressions text,
    modifier_interactions text,
    businessdate date,
    orderdateutc text,
    orderdatelocal timestamp without time zone,
    frequentcustomerid text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the ngeModifierUpsellRecommendationsToGAS mapping data flow inside the ModifierUpsellRecommendationsToGAS pipeline, which joins gasOrdersCosmos order documents against fact.transactionheader and fact.modifier_recommendations directly (bypassing the Bronze/silver staging path used by the main orders pipeline).

This is a legacy/parallel ingestion path retained for the modifier-recommendation subject area; it is not read by any of the fact.usp_* stored procedures reviewed, so its downstream consumer (if any) lives inside the data flow itself rather than in PL/pgSQL.

3.28 stg.recommendations

Landing copy of upsell recommendation events built directly from Cosmos DB order documents — the direct source for fact.usp_item_recommendations_stage_to_fact.

DDL

```
CREATE TABLE stg.recommendations (
    transactionheaderid character varying(50) NOT NULL,
    locationid character varying(50) NOT NULL,
    recommendationid character varying(50) NOT NULL,
    offereditems text,
    selecteditems text,
    prompttimestamp text,
    sysinserttime timestamp without time zone,
    syscosmosts bigint
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	transactionheaderid_recommendationid_uidx	transactionheaderid, recommendationid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the ngeUpsellRecommendationsToGAS mapping data flow inside the UpsellRecommendationsToGAS pipeline, which joins gasOrdersCosmos order documents against fact.recommendations and fact.transactionheader directly.

fact.usp_item_recommendations_stage_to_fact reads this table and, using a NOT EXISTS anti-join against fact.recommendations, inserts new recommendation rows into fact.recommendations.

3.29 stg.sent_surveys

Landing copy of which occasion surveys were sent for which orders, built directly from Cosmos DB order-feedback documents.

DDL

```
CREATE TABLE stg.sent_surveys (
    organizationid text,
    locationid text NOT NULL,
    ordersessionid text NOT NULL,
    orderid text,
    gem_event_category text,
    gem_event_type text,
    survey_metadata text,
    is_responded boolean,
    gem_event_instant text,
    gem_syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	sent_surveys_ordersessionid_pkey	locationid, ordersessionid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the ngFactOccasionSurveyDetail mapping data flow inside the OccasionSurveyOrdersFeedback pipeline, which joins ngOrdersFeedback documents against dim.occasionsurvey, dim.organizationlocation and fact.transactionheader.

This feeds fact.sent_surveys (via the data flow's own sink logic); fact.usp_gem_sent_surveys_to_fact separately maintains fact.sent_surveys from stg.silver_kiosk_events, so this data-flow path and the kiosk-events path are two independent contributors to the same fact table for different survey-delivery channels.

3.30 stg.silver_all_gem_events

Raw landing table for every GEM (device/kiosk telemetry and event) JSON record read out of the hourly Bronze ADLS partition, before it has been split out by event type. This is the single multiplexed inbox for the events pipeline.

DDL

```
CREATE TABLE stg.silver_all_gem_events (
    id text COLLATE pg_catalog."default",
    application text COLLATE pg_catalog."default",
    companyid text COLLATE pg_catalog."default",
    locationid text COLLATE pg_catalog."default",
    eventmodule text COLLATE pg_catalog."default",
    eventcategory text COLLATE pg_catalog."default",
    eventtype text COLLATE pg_catalog."default",
    severity text COLLATE pg_catalog."default",
```

```
    token text COLLATE pg_catalog."default",
    eventinstant text COLLATE pg_catalog."default",
    username text COLLATE pg_catalog."default",
    userid text COLLATE pg_catalog."default",
    device text COLLATE pg_catalog."default",
    devicename text COLLATE pg_catalog."default",
    summary text COLLATE pg_catalog."default",
    data text COLLATE pg_catalog."default",
    syscosmosticks bigint,
    syscosmosts bigint,
    bronze_folderpath text COLLATE pg_catalog."default",
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_silver_all_gem_events_syscosmosts_brin	brin	syscosmosts
idx_stg_silver_all_gem_events_folderpath	btree	"bronze_folderpath"

How It's Populated

Populated by a Copy Activity ('EventsToGAS') inside BronzeEventsToSilverAndGAS (and the legacy BronzeEventsToGAS), which reads the NDJSON files under the Bronze events/... partition path (parameterized as @dataset().p_ds_partition_path) for the current hour identified via etl.bronze_partition_registry, and bulk-loads them as-is into this table alongside bronze_folderpath and a syscosmosts column.

Once loaded, fact.usp_distribute_silver_gem_events(p_partition_path) fans this table out: it reads DISTINCT application/eventmodule/eventtype combinations for the given partition and, using NOT EXISTS anti-joins keyed on (locationid, id) or (locationid, token), inserts the rows that are new into the appropriate per-entity table — stg.silver_kiosk_events (kiosk/session events) or stg.silver_cep_incidents (connector/CEP incident events) — so the same raw row is never distributed twice.

After successful distribution, the procedure TRUNCATEs/DELETEs the processed rows from this table (or the specific bronze_folderpath) so it only ever holds the current in-flight batch, keeping it small between pipeline runs.

Indexed for this workload with a BRIN index on syscosmosts (cheap range scans over an append-then-flush table) and a b-tree on bronze_folderpath for the partition-scoped DELETE at the end of each run.

3.31 stg.silver_all_transaction_entities

Raw landing table for every order/transaction JSON record read out of the hourly Bronze ADLS orders partition, before it has been split out by document type (header, item, payment, modifier, etc.). The order-side counterpart to stg.silver_all_gem_events.

DDL

```
CREATE TABLE stg.silver_all_transaction_entities (
```

```
    -- ── Cosmos / system metadata ──────────────────────────────────────
"_attachments" TEXT,
    "_etag" TEXT,
    "_rid" TEXT,
    "_self" TEXT,
    "_lsn" BIGINT,
    "_ts" BIGINT,
    -- ── Order header scalars ──────────────────────────────────────────── "id" TEXT,
    "orderId" TEXT,
    "locationId" TEXT,
    "businessDate" TEXT,
    "orderDate" TEXT,
    "orderType" TEXT,
    "orderTypeLabel" TEXT,
    "channel" INTEGER,
    "conceptId" TEXT,
    "conceptName" TEXT,
    "kioskSessionId" TEXT,
    "clientIpAddress" TEXT,
    "guestCount" INTEGER,
    "gusetCheckImageLink" TEXT,
    -- preserved source typo "isFailedToSendToPos" BOOLEAN,
    "isTestOrder" BOOLEAN,
    "posSubmissionStatus" INTEGER,
    "originalTransactionId" TEXT,
    "refundTransactionId" TEXT,
    "refundType" TEXT,
    "refundedAmount" NUMERIC(12,3),
    "rawResponse" TEXT,
    "receiptImage" TEXT,
    "orderReceiptUrl" TEXT,
    "orderReceiptPdfUrl" TEXT,
    "type" TEXT,
    -- ── Loyalty scalars ──────────────────────────────────────────────
"loyaltyProviderTransactionId" TEXT,
    "loyaltyProviderPaymentTransactionId" TEXT,
    -- ── Complex / nested → TEXT ──────────────────────────────────────── "kioskSource"
TEXT,
    "orderIdentity" TEXT,
    "loyaltyUser" TEXT,
    "localCurrencyDetails" TEXT,
    "totals" TEXT,
    "totalsCents" TEXT,
    "receiptDetails" TEXT,
    "concepts" TEXT,
    "items" TEXT,
    "combos" TEXT,
    "discounts" TEXT,
    "paymentDetails" TEXT,
    "redeemedRewards" TEXT,
    "upsellInformation" TEXT,
    --New Fields-- "resolvedAt" TEXT,
    "thirdPartyOrderId" TEXT,
    -- ── Pipeline metadata ─────────────────────────────────────────────
"bronze_folderpath" TEXT,
    "sysinserttime" TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_all_trxn_ent_folderpath_type	btree	"bronze_folderpath", "type"

How It's Populated

Populated by the 'OrdersToGAS' Copy Activity inside BronzeOrdersToSilverAndGAS (and the legacy BronzeOrdersToGAS), which reads the Bronze orders/... partition for the current hour (again driven by etl.bronze_partition_registry) and bulk-loads the raw documents into this wide, ~53-column landing table tagged with bronze_folderpath and type.

fact.usp_distribute_silver_transaction_entities(p_partition_path) is the single large (~52,000-character) procedure that fans this table out to 10 target staging tables — silver_transaction_header, silver_transaction_item, silver_transaction_payment, silver_transaction_combo_items, silver_item_modifiers, silver_upsell_recommendations, silver_modifier_recommendations, silver_modifier_impressions, silver_modifier_interactions, silver_transaction_refunds — filtering rows by the type column and using a NOT EXISTS anti-join against each target (keyed on locationid + transactionheaderid, plus the entity's own natural key) so re-running the same partition is idempotent.

The composite index idx_stg_silver_all_trxn_ent_folderpath_type on (bronze_folderpath, type) is purpose-built for this proc's per-type filtered reads and for the end-of-run DELETE by bronze_folderpath.

Once distributed, rows for the partition are deleted from this table so it does not accumulate; etl.truncate_silver_staging() is not used for this particular table (it is cleared inside the distribution procedure itself), unlike most of the per-entity silver_* tables below.

3.32 stg.silver_cep_incidents

Per-incident staging row for GEM Connector/CEP (kiosk connectivity, hardware, ordering-flow error) events — the direct source for fact.cep_incidents.

DDL

```
CREATE TABLE stg.silver_cep_incidents (
    id text,
    application text,
    companyid text,
    locationid text,
    eventmodule text,
    eventcategory text,
    eventtype text,
    severity text,
    token text,
    eventinstant text,
    username text,
    userid text,
    device text,
    devicename text,
    summary text,
    data text,
    syscosmosticks bigint,
    syscosmosts bigint,
    silver_transform_time text,
```

```
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_cep_incidents_id_loc	btree	locationid, id

How It's Populated

Populated by fact.usp_distribute_silver_gem_events(), which routes connector/CEP-module rows out of stg.silver_all_gem_events into this table using a NOT EXISTS anti-join on (locationid, id).

fact.usp_silver_cep_incidents_to_fact_cep_incidents reads new rows here (filtered by a fact.watermarktable syscosmosts checkpoint), joins to dim.organizationlocation and fact.gem_failed_order_job_notifications for enrichment, and upserts into fact.cep_incidents.

Indexed on (locationid, id) to support the anti-join insert path from the distribution procedure. Cleared by etl.truncate_silver_staging() on the same 10-minute window as the other GEM event staging tables.

3.33 stg.silver_item_modifiers

Per-item-modifier-selection staging row (modifier group/modifier chosen on a line item) distributed out of the raw Bronze order feed — the direct source for fact.itemmodifier.

DDL

```
CREATE TABLE stg.silver_item_modifiers (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    frequentcustomerid text,
    orderitemid text,
    itemsessionid text,
    menuitemid text,
    menu_item_pos_id text,
    itemname text,
    categoryid text,
    categoryname text,
    category_pos_id text,
    itemquantity integer,
```

```
    usd_itemunitprice numeric(12,3),
    usd_total_item_price numeric(12,3),
    cents_itemunitprice bigint,
    cents_total_item_price bigint,
    items_discount_id text,
    is_items_discount_hidden_on_receipt boolean,
    items_discounts text,
    items_upsell_source text,
    items_reward_source text,
    items_special_request text,
    items_concept_id text,
    items_concept_name text,
    options_modifierid text,
    options_modifier_pos_id text,
    options_modifiername text,
    options_modifier_code text,
    options_modifiergroupid text,
    options_modifiergroupname text,
    options_modifiergroup_pos_id text,
    options_modifierquantity integer,
    options_modifierunitprice numeric(12,3),
    options_total_modifierprice numeric(12,3),
    modifier_freequantity integer,
    is_modifier_invisible boolean,
    is_modifier_default boolean,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_item_mod_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts item-modifier documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.34 stg.silver_kiosk_events

Per-kiosk-session-event staging row — the single richest events staging table, feeding dim.element, dim.view, dim.ordertype, fact.deviceevent, fact.userbehaviour, fact.sent_surveys, fact.usercheckedin, fact.ordertiming and the aborted-order recovery logic, all keyed off the kiosk session token.

DDL

```
CREATE TABLE stg.silver_kiosk_events (
    id text,
    application text,
    companyid text,
    locationid text,
    eventmodule text,
    eventcategory text,
    eventtype text,
    severity text,
    token text,
    eventinstant text,
    username text,
    userid text,
    device text,
    devicename text,
    summary text,
    data text,
    syscosmosticks bigint,
    syscosmosts bigint,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_kiosk_events_id_loc	btree	locationid, id
ix_silver_kiosk_events_syscosmosts_brin	brin	syscosmosts

How It's Populated

Populated by fact.usp_distribute_silver_gem_events(p_partition_path), which reads rows from stg.silver_all_gem_events belonging to the kiosk/session event module and inserts the ones that don't already exist here (NOT EXISTS anti-join on locationid, id), so a reprocessed Bronze partition never double-inserts.

This is the busiest staging table in the warehouse by consumer count: fact.usp_silver_kiosk_events_to_fact_deviceevent, fact.usp_silver_kiosk_events_to_fact_userbehaviour, fact.usp_gem_ordertiming_to_fact_ordertiming, fact.usp_gem_sent_surveys_to_fact, fact.usp_gem_usercheckedin_to_fact_usercheckedin, fact.usp_silver_aborted_orders_and_items_to_fact, fact.usp_stg_occasionsurveydetail_to_fact, dim.usp_refresh_element, dim.usp_refresh_view and dim.usp_refresh_ordertype all read from it, each filtering on their own eventtype/eventcategory/token pattern.

The b-tree index on (locationid, id) supports the anti-join insert path, while the BRIN index on syscosmosts supports the many downstream watermark range-scans (syscosmosts > last-processed-watermark) used by nearly every consumer procedure listed above.

Cleared on the same 10-minute sliding watermark by etl.truncate_silver_staging() (a slightly longer window than the header/item tables, to make sure a kiosk session that started just before the hour boundary is still present when its completing order event arrives).

3.35 stg.silver_modifier_impressions

Per-item modifier-impression staging row (which modifiers were shown to the guest and in what position) — the direct source for fact.modifier_impressions.

DDL

```
CREATE TABLE stg.silver_modifier_impressions (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    frequentcustomerid text,
    menuitemid text,
    parentmodifierid text,
    selection_type text,
    modifier_impressions_nesting_depth integer,
    modifier_impressions_context text,
    strategy text,
    modifierid text,
    score integer,
    "position" integer,
    selected boolean,
    pre_selected boolean,
    pre_deselected boolean,
    confirmed_removed boolean,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_mod_imp_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts modifier-impression documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.36 stg.silver_modifier_interactions

Per-modifier interaction-event staging row (guest taps/selects/deselects a recommended modifier) — the direct source for fact.modifier_interactions.

DDL

```
CREATE TABLE stg.silver_modifier_interactions (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    frequentcustomerid text,
    menuitemid text,
    modifierid text,
    modifiergroupid text,
    parent_modifier_id text,
    selection_type text,
    modifier_interactions_action text,
    modifier_interactions_recorded_at text,
    modifier_interactions_nesting_depth integer,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_mod_int_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts modifier-interaction documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.37 stg.silver_modifier_recommendations

Per-order modifier-recommendation session staging row — the direct source for fact.modifier_recommendations, which is in turn read by the impression/interaction analysis procedures.

DDL

```
CREATE TABLE stg.silver_modifier_recommendations (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    frequentcustomerid text,
    modifier_interactions text,
    modifier_impressions text,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_mod_rec_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts modifier-recommendation documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.38 stg.silver_transaction_combo_items

Per-combo-component staging row linking a combo line item to its component items — feeds fact.transactionitem's combo/component enrichment logic in fact.usp_silver_transaction_item_to_fact.

DDL

```
CREATE TABLE stg.silver_transaction_combo_items (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    frequentcustomerid text,
    combo_id text,
    combo_pos_id text,
    combo_name text,
    combo_order_item_id text,
    combo_item_session_id text,
    combo_concept_id text,
    combo_concept_name text,
    cents_combo_unit_price bigint,
    cents_combo_total_price bigint,
    combo_quantity integer,
    combo_special_request text,
    combo_upsell_source text,
    combo_reward_source text,
    component_id text,
    component_pos_id text,
    component_name text,
    component_item_order_item_id text,
    component_item_menu_item_id text,
```

```
    component_item_name text,
    component_item_menu_item_pos_id text,
    component_item_session_id text,
    component_item_concept_id text,
    component_item_concept_name text,
    component_item_quantity integer,
    component_item_price numeric(12,3),
    component_item_unit_price numeric(12,3),
    component_item_cents_unit_price bigint,
    component_item_total_price numeric(12,3),
    component_item_cents_total_price bigint,
    component_item_special_request text,
    component_item_upsell_source text,
    component_item_reward_source text,
    component_item_discount_id text,
    is_component_item_discount_hidden_on_receipt boolean,
    component_item_discounts text,
    component_selections_items text,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_trxn_combo_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts combo-item documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.39 stg.silver_transaction_header

Per-order header staging row (order/session identifiers, totals, timing, order type, location) distributed out of the raw Bronze order feed — the direct source for fact.transactionheader.

DDL

```
CREATE TABLE stg.silver_transaction_header (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    channel integer,
    items_array text,
    payments_array text,
    numberofitems smallint,
    numberofpayments smallint,
    concept_id text,
    concept_name text,
    ordertype text,
    order_type_label text,
    order_completion_status text,
    pos_submission_status integer,
    is_send_to_pos_failed boolean,
    is_test_order boolean,
    frequentcustomerid text,
    customername text,
    client_ip_address text,
    order_identity_order_token text,
    order_identity_pos_order_token text,
    order_identity_phone text,
    order_identity_phone_country_code text,
    order_identity_email text,
    order_identity_table_tent text,
    order_identity_device_imei text,
    guest_count integer,
    guest_check_code text,
    genesis_fiscal_fields text,
    order_language text,
    receipt_printing_type text,
    loyalty_transaction_id text,
    loyalty_payment_transaction_id text,
    loyalty_earned_points text,
    local_currency_code text,
    local_currency_additional_info text,
    usd_amount numeric(12,3),
    usd_subtotal numeric(12,3),
    usd_tax numeric(12,3),
    usd_tip numeric(12,3),
    usd_discount numeric(12,3),
    usd_reward numeric(12,3),
    usd_service_charge numeric(12,3),
    usd_charity_amount numeric(12,3),
    cents_amount bigint,
    cents_subtotal bigint,
    cents_tax bigint,
    cents_tip bigint,
    cents_discount bigint,
    cents_reward bigint,
    cents_service_charge bigint,
    cents_charity_amount bigint,
    bronze_filepath text,
```

```
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    discounts_array text COLLATE pg_catalog."default",
    combos_array text COLLATE pg_catalog."default",
    redeemed_rewards_array text COLLATE pg_catalog."default",
    concepts_array text COLLATE pg_catalog."default",
    upsell_prompt_array text COLLATE pg_catalog."default",
    modifier_interactions_array text COLLATE pg_catalog."default",
    modifier_impressions_array text COLLATE pg_catalog."default",
    loyalty_user_object text COLLATE pg_catalog."default",
    receipt_details_object text COLLATE pg_catalog."default",
    kiosk_source_object text COLLATE pg_catalog."default",
    local_currency_details_object text COLLATE pg_catalog."default",
    order_identity_object text COLLATE pg_catalog."default",
    totals_object text COLLATE pg_catalog."default",
    totals_cents_object text COLLATE pg_catalog."default",
    bronze_folderpath text COLLATE pg_catalog."default"
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_trxn_hdr_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts transaction-header documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.40 stg.silver_transaction_item

Per-line-item staging row (item id/name, quantity, price, combo linkage) distributed out of the raw Bronze order feed — the direct source for fact.transactionitem.

DDL

```
CREATE TABLE stg.silver_transaction_item (
    transactionheaderid text,
    orderid text,
```

```
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    frequentcustomerid text,
    orderitemid text,
    itemsessionid text,
    menuitemid text,
    menu_item_pos_id text,
    itemname text,
    categoryid text,
    categoryname text,
    category_pos_id text,
    items_concept_id text,
    items_concept_name text,
    itemquantity integer,
    usd_itemunitprice numeric(12,3),
    usd_total_item_price numeric(12,3),
    cents_itemunitprice bigint,
    cents_total_item_price bigint,
    modifier_options text,
    items_discount_id text,
    is_items_discount_hidden_on_receipt boolean,
    items_discounts text,
    items_upsell_source text,
    items_reward_source text,
    items_special_request text,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_trxn_item_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts transaction-item documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.41 stg.silver_transaction_payment

Per-payment staging row (tender type, amount, payment transaction id) distributed out of the raw Bronze order feed — the direct source for fact.transactionpayment.

DDL

```
CREATE TABLE stg.silver_transaction_payment (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    payment_transactionid text,
    payment_method text,
    payment_status text,
    payment_amount numeric(12,3),
    payment_tender_id text,
    payment_integration_id text,
    payment_integration_label text,
    payment_card_name text,
    payment_card_number text,
    is_amazon_one_payment boolean,
    card_info_card_type text,
    card_info_last_four text,
    card_info_masked_card_number text,
    card_info_zip_code text,
    card_info_expiration_month text,
    card_info_expiration_year text,
    card_info_processor_auth_code text,
    card_info_available_balance numeric(12,3),
    payment_capture_details text,
    payment_settlement_details text,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_trxn_pmt_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts transaction-payment documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.42 stg.silver_transaction_refunds

Per-refund staging row (refunded amount, reason, linked payment) distributed out of the raw Bronze order feed — the direct source for fact.transactionrefunds.

DDL

```
CREATE TABLE stg.silver_transaction_refunds (
    locationid text,
    transactionheaderid text NOT NULL,
    orderid text,
    original_transaction_id text,
    refund_transaction_id text,
    refund_type text,
    refunded_amount numeric(12,3),
    order_completion_status text,
    orderdateutc text,
    syscosmosts bigint,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_trxn_ref_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts transaction-refund documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.43 stg.silver_upsell_recommendations

Per-order upsell-recommendation staging row (which items were recommended, whether accepted) — the direct source for fact.usp_silver_upsell_recommendations_to_fact, which writes into fact.recommendations.

DDL

```
CREATE TABLE stg.silver_upsell_recommendations (
    transactionheaderid text,
    orderid text,
    ordersessionid text,
    orderdateutc text,
    businessdate text,
    syscosmosts bigint,
    locationid text,
    kioskid text,
    kiosk_name text,
    kiosk_mode integer,
    is_test_order boolean,
    frequentcustomerid text,
    recommendationid text,
    prompttimestamp text,
    modal_version text,
    offered_items text,
    selected_items text,
    order_completion_status text,
    bronze_filepath text,
    silver_transform_time text,
    silver_folderpath text,
    sysinserttime timestamp without time zone,
    bronze_folderpath TEXT
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_stg_silver_upsell_rec_txid_loc	btree	locationid, transactionheaderid

How It's Populated

Populated exclusively by fact.usp_distribute_silver_transaction_entities(), which extracts upsell-recommendation documents (filtered by type) out of stg.silver_all_transaction_entities and inserts the rows that don't already exist here (NOT EXISTS anti-join on locationid, transactionheaderid), guaranteeing the fan-out is idempotent if a Bronze partition is reprocessed.

The composite index on (locationid, transactionheaderid) exists specifically to make that anti-join lookup fast for every incoming partition.

Downstream, this staging row is the direct source for the corresponding fact.usp_silver_*_to_fact() procedure, and is periodically cleared by etl.truncate_silver_staging() on a 10-minute sliding watermark window (kept just long enough to cover late-arriving kiosk sessions) inside the BronzeOrdersToSilverAndGAS / TruncateStagingTables step of the daily master run.

3.44 stg.temp_silver_transaction_header

A 40-column working/scratch copy of the transaction-header staging shape.

DDL

```
CREATE TABLE stg.temp_silver_transaction_header (
    id integer,
    transactionheaderid text,
    orderid text,
    locationid text,
    kioskid text,
    ordersessionid text,
    dateid integer,
    orderdateutc text,
    orderdatelocal timestamp without time zone,
    orderstatus text,
    ordertype integer,
    numberofitems smallint,
    numberofpayments smallint,
    ordersredeemedrewards numeric(12,3),
    ordersubtotal numeric(12,3),
    ordertotal numeric(12,3),
    ordertax numeric(12,3),
    ordertip numeric(12,3),
    orderdiscount numeric(12,3),
    orderbalance numeric(12,3),
    paymentstatus text,
    sourcefile text,
    createddate timestamp without time zone,
    charityamount numeric(12,3),
    orderservicecharge numeric(12,3),
    businessdate date,
    syscosmosts bigint,
    channel text,
    guestcount integer,
    frequentcustomerid text,
    customername text
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts references this table. The 'temp_' naming strongly suggests an ad hoc scratch table from manual debugging or a one-time backfill exercise rather than a pipeline-managed object.

3.45 stg.transactionheader

A 44-column table that duplicates much of the shape of stg.silver_transaction_header / fact.transactionheader, with its own primary key on id.

DDL

```
CREATE TABLE stg.transactionheader (
    id text NOT NULL,
    kiosksessionid text,
    orderid text,
    locationid text,
    type text,
    ordertype text,
    ordertypelabel text,
    channel integer,
    orderdate text,
    businessdate text,
    guestcount integer,
    possubmissionstatus integer,
    isfailedtosendtopos boolean,
    istestorder boolean,
    clientipaddress text,
    conceptid text,
    conceptname text,
    loyaltyuser text,
    loyaltyprovidertransactionid text,
    loyaltyproviderpaymenttransactionid text,
    receiptimage text,
    orderreceipturl text,
    orderreceiptpdfurl text,
    guestcheckimagelink text,
    totals text,
    totalscents text,
    localcurrencydetails text,
    orderidentity text,
    kiosksource text,
    upsellinformation text,
    receiptdetails text,
    items text,
    combos text,
    paymentdetails text,
    redeemedrewards text,
    discounts text,
    concepts text,
    sys_rid text,
    sys_self text,
    sys_etag text,
    sys_attachments text,
```

```
    sys_lsn bigint,
    syscosmosts bigint,
    bronze_filepath text
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	pk_transactionheader	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts targets this table. It is registered in the ADF dataset catalog (stgTransactionHeaderAnalyticsDb) but that dataset is not referenced by any activity in any of the 67 pipelines reviewed, so it appears to be a retired or planned-but-unused object.

4. DIM Schema - Dimension Tables

Forty-one dimension tables describing organizations, locations, kiosks, menu items, modifiers, order types and related reference data.

4.1 dim.abtests

One row per A/B test variant assignment/exposure for an order session (experiment, variant, device) — used to slice fact metrics by experiment arm.

DDL

```
CREATE TABLE dim.abtests (
    abtestid bigint NOT NULL,
    organizationid text,
    locationid text,
    experimentid text,
    experimentname text,
    variantid text,
    variantname text,
    ordersessionid text,
    deviceid text,
    devicename text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated directly by the 'dimABTests' mapping data flow inside the DimensionABTests pipeline (part of the GEM_GSH_GOU_Master chain), which reads GEM event documents and writes/updates rows here based on experiment-exposure events.

fact.usp_update_datetime_fields subsequently reads dim.abtests (joined with dim.organization) to backfill business-local date/time fields on fact.transactionheader for orders tied to an experiment.

No stored procedure directly targets this table with INSERT/UPDATE in the reviewed PL/pgSQL source; all of its writes happen inside the ADF data flow itself.

4.2 dim.catalog

One row per GMS product catalog (master/standalone/ECM-enabled flags, owning organization) — the top of the menu hierarchy referenced by dim.itemcategory, dim.menuitem, dim.modifier and dim.modifier_group via catalogid.

DDL

```
CREATE TABLE dim.catalog (
    catalogid character varying(50) NOT NULL,
    catalogname character varying(255),
    organizationid character varying(40),
    is_catalog_deleted boolean,
    catalog_created_on timestamp without time zone,
    catalog_modified_on timestamp without time zone,
    gem_company_id character varying(255),
    gem_location_id character varying(255),
    is_sync_in_progress boolean,
    is_standalone boolean,
    is_master boolean,
    is_ecm_enabled boolean,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`catalog_pkey`	`catalogid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_catalog is called by DimensionOrganizationLocationCatalog::dim_usp_refresh_catalog (part of the daily DimensionUpdateOrdersToAnalyticsDb / DimensionMaster chain).

It reads stg.dim_catalog (itself a full Copy-Activity snapshot of GMS public.item_master catalog rows), dedups with DISTINCT ON (catalogid), and uses a NOT EXISTS anti-join against dim.catalog to insert brand-new catalogs, followed by an UPDATE of name/organization/sync-flag columns for catalogs that already exist — a classic insert-new-then-update-changed pattern rather than a single ON CONFLICT upsert.

4.3 dim.category_hierarchy

One row per (location, category, menu item) combination describing the menu category tree — bridges dim.itemcategory and dim.menuitem for reporting.

DDL

```
CREATE TABLE dim.category_hierarchy (
    organizationid text,
    locationid text NOT NULL,
    mapping_created_on timestamp without time zone,
    mapping_modified_on timestamp without time zone,
    is_mapping_active boolean,
    is_mapping_deleted boolean,
    catalogid text,
    catalogname text,
    catalog_created_on timestamp without time zone,
    catalog_modified_on timestamp without time zone,
    is_catalog_active boolean,
    is_catalog_deleted boolean,
    categoryid text NOT NULL,
    categoryname text,
    category_created_on timestamp without time zone,
    category_modified_on timestamp without time zone,
    is_category_active boolean,
    is_category_deleted boolean,
    menuitemid text,
    entitytype text,
    item_class_type integer,
    menuitemname text,
    item_created_on timestamp without time zone,
    item_modified_on timestamp without time zone,
    is_item_active boolean,
    is_item_deleted boolean,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	`location_category_menuitem_unq`	`locationid, categoryid, menuitemid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_category_hierarchy is called by DimensionMenuItems::dim_usp_refresh_category_hierarchy.

It reads stg.dim_category_hierarchy (a full Copy-Activity snapshot of GMS category/menu-item hierarchy rows), dedups with DISTINCT ON (locationid, categoryid, menuitemid), and performs a NOT EXISTS-guarded insert of new combinations plus an UPDATE of mapping_modified_on/is_mapping_active/is_mapping_deleted for existing ones.

Also fed historically by the NgeCategoryHierarchy and gmsMenuItemsToGAS data flows and by a ChildPipTrainingDataML Copy Activity (CopyCategoryHierarchy) used specifically to prepare a snapshot for ML training extracts — these are parallel/legacy paths alongside the stored-procedure path above.

4.4 dim.company

One row per company (the tenant/brand entity that owns one or more locations) — top of the organizational hierarchy alongside dim.organization/dim.location.

DDL

```
CREATE TABLE dim.company (
    companyid text NOT NULL,
    companyname text NOT NULL,
    businessemail text,
    businessphone text,
    businesstype text,
    address1 text,
    address2 text,
    city text,
    state text,
    zipcode text
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	company_pkey	companyid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated directly by the 'CosmosCompany' mapping data flow, sourced from the gasCosmosCompany Cosmos DB container. No stored procedure targets this table; the data flow performs its own upsert logic.

4.5 dim.datedim

Standard calendar date dimension (one row per hour: dateid in YYYYMMDDHH format, date/day/week/month/quarter/year attributes, day-part label) used to join business-local timestamps across nearly every fact table and to drive etl.bronze_partition_registry's partition generation.

DDL

```
CREATE TABLE dim.datedim (
    dateid integer NOT NULL,
    datets timestamp without time zone NOT NULL,
    hourofday text NOT NULL,
    dayval date NOT NULL,
    daynum smallint NOT NULL,
    dayname text NOT NULL,
    weekval integer NOT NULL,
    monthval integer NOT NULL,
    monthname text NOT NULL,
    quarterval integer NOT NULL,
    yearval integer NOT NULL,
    daypart text NOT NULL,
    dayofyear smallint NOT NULL
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	datedim_pk	dateid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
IX_dateid_datets	btree	dateid, datets
idx_datedim_dateid	btree	dateid
idx_datedim_datets	btree	datets

How It's Populated

Not populated by any ADF pipeline activity or stored procedure in the reviewed artifacts — this is a classic date-dimension table, generated once (and periodically extended forward) via a one-time SQL script (a generate_series over dateid/hour combinations), then left essentially static and simply read from by every consumer.

Three indexes support the very high join volume against this table: a composite (dateid, datets), and single-column indexes on dateid and datets.

4.6 dim.device

One row per physical kiosk/device (device id, location, company, type, state, test-mode flag) — the device-health counterpart to dim.kiosk.

DDL

```
CREATE TABLE dim.device (
    id bigint NOT NULL,
    deviceid character varying(50) NOT NULL,
    devicetype character varying(50) NOT NULL,
    devicename text,
    locationid character varying(50) NOT NULL,
    companyid character varying(50) NOT NULL,
    currentversion character varying(50),
    ipaddress character varying(50),
    state character varying(50) NOT NULL,
    previousstate character varying(50),
    statechangedate timestamp without time zone,
    enrollmentdate timestamp without time zone NOT NULL,
    disenrollmentdate timestamp without time zone,
    disenrollmentreason text,
    testmode boolean DEFAULT false
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	device_deviceid_locationid_companyid_key	deviceid, locationid, companyid
PRIMARY KEY	device_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
deviceid_locationid_companyid_idx	btree	deviceid, locationid, companyid
idx_device_id	btree	deviceid
idx_device_location_id	btree	locationid
idx_device_state	btree	state
idx_device_testmode	btree	testmode

How It's Populated

Populated directly by the DevicesAndPeripherals and gshDevicesAndPeripheralsToAnalyticsDb data flows, both sourced from gsh.device (the GSH device-health platform's own device table) alongside gsh.peripheral/gsh.devicehealth/gsh.peripheralhealth for related enrichment written to fact.devicehealth and fact.peripheralhealth in the same data flow run.

No PL/pgSQL procedure targets this table directly; five indexes (composite deviceid+locationid+companyid, plus single-column deviceid, locationid, state, testmode) support the very frequent lookups against it from fact and dim procedures (e.g. dim.usp_grubbrr_install_base joins dim.kiosk/dim.organization, and several fact procs filter by device state/test mode).

4.7 dim.duplicate_items_master

One row per (location, category, menu item) identifying a canonical 'master' item id where GMS has created duplicate item records for the same logical menu item — used to collapse duplicates in reporting.

DDL

```
CREATE TABLE dim.duplicate_items_master (
    organizationid text,
    locationid text NOT NULL,
    categoryid text NOT NULL,
    categoryname text,
    menuitemid text,
    entitytype text,
    item_class_type integer,
    menuitemname text,
    instance_count integer,
    masteritemid text,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	`locationid_categoryid_menuitemid_unq`	`locationid, categoryid, menuitemid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_master_keys_for_duplicate_items is called by RefreshMLTables::MasterKeysForDuplicateItems (and the ChildPipTrainingDataML equivalent), i.e. it runs as part of the daily ML-tables refresh chain, not the main Dimensions/Facts chain.

It TRUNCATEs this table and rebuilds it in full from dim.category_hierarchy and fact.transactionitem (using a NOT EXISTS check to only keep hierarchy rows that actually have matching transaction volume), making it a full-refresh rather than incremental table.

4.8 dim.element

Small lookup of GEM UI 'element' identifiers (button/screen element clicked) referenced by fact.userbehaviour for clickstream analysis.

DDL

```
CREATE TABLE dim.element (
    elementid integer DEFAULT nextval('dim.element_elementid_seq'::regclass) NOT NULL,
    sourceelementid text,
    elementname text,
    sysinserttime timestamp without time zone,
```

```
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	dimelement_pkey	elementid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_element is called by GEM_GSH_GOU_Master::dimElement, immediately before dimView in that chain.

It reads distinct new element values out of stg.silver_kiosk_events (NOT EXISTS-guarded against dim.element) and inserts them — a simple append-only lookup-discovery pattern with no updates.

Also referenced (read-only) by the gemUserBehaviourEventsToFact family of data flows when building fact.userbehaviour.

4.9 dim.experiment

Two-column definition table for A/B test experiments (dimkey surrogate key), presumably referenced conceptually by dim.abtests' experimentid/experimentname columns.

DDL

```
CREATE TABLE dim.experiment (
    dimkey integer NOT NULL,
    data jsonb
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	experiment_pkey	dimkey

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts targets this table. dim.abtests carries its own experimentid/experimentname text columns directly (denormalized) rather than joining to dim.experiment, so this table appears to be an unused or not-yet-wired-up dimension — possibly intended for a future normalized redesign of the A/B testing model.

4.10 dim.feedbackrating

Two-column static lookup mapping a numeric feedback rating to its label — used to decode fact.itemssurvey / fact.occasionsurveydetail rating values in reporting.

DDL

```
CREATE TABLE dim.feedbackrating (
    rating text,
    ratingdesc text
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
rating_idx	btree	rating

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table. It is a small, static reference/seed table (rating codes rarely if ever change), most likely populated once by hand and not part of the recurring pipeline.

4.11 dim.feedbackstatus

Two-column static lookup mapping a survey-transaction status code to its label — used to decode fact.itemssurvey / fact.occasionsurveydetail status values.

DDL

```
CREATE TABLE dim.feedbackstatus (
    surveytransstatus text,
    statusdesc text
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
surveytransstatus_idx	btree	surveytransstatus

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table. Like dim.feedbackrating, it is a static reference/seed table maintained outside the regular pipeline.

4.12 dim.frequentcustomer

One row per loyalty/frequent-customer profile (name, contact info, order count, last order date) — joined into fact.usp_customer_menu_preferences and fact.usp_location_menu_preferences for personalization analytics.

DDL

```
CREATE TABLE dim.frequentcustomer (
    customerkey bigint DEFAULT nextval('dim.frequentcustomer_customerkey_seq'::regclass)
NOT NULL,
    frequentcustomerid text NOT NULL,
    firstname text,
    lastname text,
    email text,
    phone text,
    source text,
    organizationid text,
    createddate text,
    lastorderdate text,
    ordercount integer DEFAULT 0 NOT NULL,
    amountspent numeric DEFAULT 0 NOT NULL,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`frequent_customer_pk`	`frequentcustomerid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_frequentcustomer is called by DimensionFrequentCustomer::'Update OrderCount and AmtSpent' (part of the DimensionMaster chain, run independently of DimensionMenuItems/DimensionMenuModifiers/DimensionKiosks).

It reads stg.dim_frequentcustomer (a full Copy-Activity snapshot from the NGE loyalty ngeFrequentCustomer container), dedups with DISTINCT ON (frequentcustomerid), and does a NOT EXISTS-guarded insert of new customers plus an UPDATE of contact/order-count attributes for existing ones. The pipeline's own DimFrequentCustomer data flow additionally maintains order-count/amount-spent aggregates directly.

4.13 dim.frequentcustomer_bkp

Structural backup copy of dim.frequentcustomer (same primary key, same shape).

DDL

```
CREATE TABLE dim.frequentcustomer_bkp (
    customerkey bigint NOT NULL,
    frequentcustomerid text NOT NULL,
    firstname text,
    lastname text,
    email text,
    phone text,
    source text,
    organizationid text,
    createddate text,
    lastorderdate text,
    ordercount integer DEFAULT 0 NOT NULL,
    amountspent numeric DEFAULT 0 NOT NULL,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`frequentcustomer_bkp_pk`	`frequentcustomerid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table. The '_bkp' suffix and identical shape to dim.frequentcustomer strongly indicate a manual point-in-time backup snapshot taken before a migration, bulk update, or schema change, rather than an actively refreshed pipeline table.

4.14 dim.grubbrr_source_lookup

Small (3-column) static lookup, by naming convention mapping internal source-system codes to labels used elsewhere (e.g. dim.frequentcustomer.source, fact table 'source'/'sourceid' columns).

DDL

```
CREATE TABLE dim.grubbrr_source_lookup (
    id integer NOT NULL,
    source text,
    description text
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	grubbrr_source_lookup_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table in the reviewed artifacts — a static reference/seed table maintained outside the recurring pipeline.

4.15 dim.holidays

Static holiday calendar (8 columns) presumably used for business-day/holiday flags in reporting and possibly as an input to demand-forecasting features.

DDL

```
CREATE TABLE dim.holidays (
    holiday_name character varying(50),
    celebrated_date timestamp without time zone,
    month integer,
    day integer,
    holiday_type character varying(20),
    religion character varying(20),
    is_public boolean,
    is_dynamic boolean
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table in the reviewed artifacts. Holiday calendars are typically seeded once per year from an external reference and rarely change intra-year, consistent with it being maintained outside the automated pipeline.

4.16 dim.item_modifier_group_modifier_mapping

Wide (25-column) mapping of menu item -> modifier group -> modifier, the fully denormalized version of the item/modifier hierarchy used as an ML feature source.

DDL

```
CREATE TABLE dim.item_modifier_group_modifier_mapping (
    catalogid character varying(50) NOT NULL,
    menuitemid character varying(50) NOT NULL,
    modifiergroupid character varying(50) NOT NULL,
    modifierid character varying(50) NOT NULL,
    itm_modgrp_min_selection integer,
    itm_modgrp_max_selection integer,
    itm_modgrp_free_count integer,
    is_itm_modgrp_active boolean,
    is_itm_modgrp_deleted boolean,
    itm_modgrp_created_on timestamp without time zone,
    itm_modgrp_modified_on timestamp without time zone,
    is_itm_modgrp_invisible boolean,
    is_default boolean,
    min_quantity integer,
    max_quantity integer,
    allow_quantity_increment boolean,
    increment_step integer,
    default_quantity integer,
    is_modgrp_modfr_active boolean NOT NULL,
    is_modgrp_modfr_deleted boolean NOT NULL,
    modgrp_modfr_created_on timestamp without time zone,
    modgrp_modfr_modified_on timestamp without time zone,
    is_modgrp_modfr_invisible boolean,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	item_modgrp_modfr_unq	menuitemid, modifiergroupid, modifierid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by Copy Activities across three pipelines — 'CopyItemModfrMapping' in ChildPipTrainingDataML and 'CopyModfrMapping' in both DimensionMasterItemsAndModifiers and DimensionMenuModifiers — all sourced from GMS public.item_master.

No PL/pgSQL stored procedure targets this table directly in the reviewed source; it is read (not written) by ml.usp_refresh_item_modifiergroup_modifier_mapping as one of its JOIN inputs when building ml.item_modifiergroup_modifier_mapping.

4.17 dim.itemcategory

One row per (location, category) — menu category attributes (name, active/deleted flags, item/combo counts) — parent of dim.category_hierarchy and dim.itemcategorymapping.

DDL

```
CREATE TABLE dim.itemcategory (
    id bigint DEFAULT nextval('dim.itemcategory_id_seq'::regclass) NOT NULL,
    locationid text NOT NULL,
    categoryid text NOT NULL,
    categoryname text NOT NULL,
    isactive boolean,
    catalogid text,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    is_category_deleted boolean,
    category_created_on timestamp without time zone,
    category_modified_on timestamp without time zone,
    is_alcoholic boolean,
    number_of_items smallint,
    number_of_sub_categories smallint,
    number_of_item_variations smallint,
    number_of_combos smallint,
    number_of_combo_families smallint
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	itemcategory_pk	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
itemcategory_idx	btree (unique)	locationid, categoryid
itemcategory_locationid_idx	btree	locationid

How It's Populated

dim.usp_refresh_itemcategory is called by DimensionMenuItems::dim_usp_refresh_itemcategory.

It reads stg.dim_itemcategory (a full Copy-Activity snapshot of GMS category records), dedups with DISTINCT ON (locationid, categoryid), and performs a NOT EXISTS-guarded insert of new categories plus an UPDATE of name/active-flag/count attributes for existing ones.

Also fed historically by the CosmosOrdersItemsCategories data flow, a parallel/legacy Cosmos-direct path alongside the GMS-based stored-procedure path above.

4.18 dim.itemcategory_bkp

Structural backup copy of (an earlier, narrower 5-column version of) dim.itemcategory.

DDL

```
CREATE TABLE dim.itemcategory_bkp (
    id bigint NOT NULL,
    locationid text NOT NULL,
    categoryid text NOT NULL,
    categoryname text NOT NULL,
    isactive boolean DEFAULT true NOT NULL
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	itemcategory_bkp_pk	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table. Like the other '_bkp' tables, it is almost certainly a manual point-in-time snapshot rather than an actively maintained pipeline object.

4.19 dim.itemcategorymapping

9-column mapping table, by naming and shape a narrower predecessor or alternate view of dim.category_hierarchy / dim.itemcategory relationships.

DDL

```
CREATE TABLE dim.itemcategorymapping (
    categoryid character varying(50) NOT NULL,
    menuitemid character varying(50),
    subcategoryid character varying(50),
    isactive boolean,
    isdeleted boolean,
    modifiedon timestamp without time zone,
    locationid text,
    categoryname text,
    menuitemname text
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts references this table (it does have an ADF dataset registered — dimItemCategoryMapping — but that dataset is never used as a source or sink by any activity in any of the 67 pipelines reviewed). It appears to be a retired or planned-but-unused object; recommend confirming with the team before depending on it.

4.20 dim.kiosk

One row per (location, kiosk) — kiosk name, app version, device type, test-kiosk flag — the ordering-platform counterpart to the device-health dim.device/dim.kioskdetails tables.

DDL

```
CREATE TABLE dim.kiosk (
    id bigint DEFAULT nextval('dim.kiosk_id_seq'::regclass) NOT NULL,
    locationid text NOT NULL,
    kioskid text NOT NULL,
    kioskname text,
    serialnumber text,
    appversion text,
    istestkiosk boolean,
    devicetype character varying(50) DEFAULT 'kiosk'::character varying NOT NULL,
    devicecreatedon timestamp without time zone,
    devicedeletedon timestamp without time zone,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	kiosk_pk	id
UNIQUE	kiosk_uidx	locationid, kioskid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_kiosk is called by DimensionKiosks::dim_usp_refresh_kiosk.

It reads stg.dim_kiosk (a full Copy-Activity snapshot of gsh.device attributes), dedups with DISTINCT ON (locationid, kioskid), and performs a NOT EXISTS-guarded insert plus attribute UPDATE.

The gshDeviceToDimKiosk data flow (inside DimensionUpdateOrdersToAnalyticsDb, immediately before LoadOrderTypes) separately keeps this table in sync directly from gsh.device as part of the daily Dimensions run.

4.21 dim.kioskdetails

One consolidated row per location (42 columns) merging kiosk appearance, config, POS provider, payment provider and loyalty configuration — the primary source for the Grubbrr install-base dashboard views.

DDL

```
CREATE TABLE dim.kioskdetails (
    id text,
    locationid text NOT NULL,
    kiosks text,
    devicetype text,
    syncversion text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    pos_provider text,
    loyalty_provider text,
    payment_provider text,
    scanners text,
    item_special_request text,
    legal_copy_enabled boolean,
    ada_configuration text,
    calculate_default_modifier_price boolean,
    track_kiosk_user_behavior boolean,
    loyalty_feature boolean,
    pickup_flow boolean,
    pos_auto_applied_discount boolean,
    search_functionality_enabled boolean,
    recent_orders_enabled boolean,
    play_card_config text,
    round_up_for_charity boolean,
    calories_enabled boolean,
    scan_and_go_enabled boolean,
    age_verification text,
    tips_settings text,
    business_hours_config text,
    order_types text,
    localization text,
    kiosk_receipt_settings text,
    kiosk_fonts text,
    kiosk_appearance_text_overrides text,
    kiosk_appearance_style_options text,
    loyalty_display_settings text,
    preorder_popup_enabled boolean,
    preorder_popup_text text,
    disclaimer_text text,
    order_limit_config text,
    menu_behavior_config text,
    perform_pos_status_check boolean,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`locationid_pkey`	`locationid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_dim_location_kiosk_details (called by GrubbrrInstallBaseDashboard::dim_usp_grubbrr_install_base, on the TwiceDaily trigger) and its near-duplicate dim.usp_refresh_location_kiosk_details both LEFT JOIN the six install-base staging tables — stg.dim_location_kiosks (base), stg.dim_pos_provider, stg.dim_loyalty_configuration, stg.dim_payment_provider, stg.dim_kiosk_config, stg.dim_kiosk_appearance — deduping each side with its own DISTINCT ON, and perform a single INSERT ... ON CONFLICT (locationid) DO UPDATE into this table.

The GrubbrrInstallBaseDashboard / NewGrubbrrInstallBaseDashboard data flows independently write to this table as well, joining several Cosmos DB containers (gasCosmosLocation, cosmosConnectorConfiguration, ngeLoyaltyConfigurations) directly — a parallel enrichment path alongside the stored-procedure merge described above.

This table is itself the primary source, together with dim.organizationlocation, dim.kiosk and dim.organization, for dim.usp_grubbrr_install_base and dim.usp_grubbrr_install_base_all_devices, which build the two reporting views dim.vw_grubbrrinstallbase and dim.vw_grubbrrinstallbase_all_devices.

4.22 dim.location

One row per (company, location) — address, timezone, location group — the physical-store dimension joined by almost every fact table via locationid.

DDL

```
CREATE TABLE dim.location (
    locationid text NOT NULL,
    companyid text NOT NULL,
    locationgroupid text,
    locationname text NOT NULL,
    address1 text,
    address2 text,
    city text,
    state text,
    zipcode text,
    latitude text,
    longitude text,
    timezone text,
    sysinserttime TIMESTAMP,
    sysupdatetime TIMESTAMP
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	location_pk	companyid, locationid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
dim_location_idx	btree	locationid
locationgroupid_idx	btree	locationgroupid

How It's Populated

dim.usp_refresh_organizationlocation (called by DimensionOrganizationLocationCatalog::dim_usp_refresh_organizationlocation) also maintains dim.location as a side effect: after upserting dim.organizationlocation, it uses two NOT EXISTS-guarded inserts to backfill dim.location rows sourced from dim.organization's address/city/state/zip/timezone columns joined to the organization-location mapping.

The CosmosLocations data flow (inside DimensionUpdateOrdersToAnalyticsDb's 'Locations' step, run right after OrderItemCategories) is the primary direct writer, sourced from the gasCosmosLocation Cosmos DB container.

Two indexes — a single-column locationid and locationgroupid — support the extremely high join fan-out from fact tables.

4.23 dim.locationcatalog

One row per (organization, location) recording which catalog is assigned to that location.

DDL

```
CREATE TABLE dim.locationcatalog (
    id bigint NOT NULL,
    organizationid text NOT NULL,
    locationid text NOT NULL,
    locationname text,
    catalogid text,
    timezone text,
    menuid text
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	location_ctlg_pk	organizationid, locationid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated directly by the 'dimLocationCatalog' mapping data flow, sourced from the gasCosmosLocation container. No stored procedure targets this table in the reviewed PL/pgSQL source.

4.24 dim.menuentities

27-column dimension describing menu 'entities' (a GMS concept spanning items/combos/modifiers as a unified entity type) — parent-level table referenced by ml.usp_refresh_menu_entities's naming convention.

DDL

```
CREATE TABLE dim.menuentities (
    catalogid text NOT NULL,
    entityid text NOT NULL,
    entitytype text NOT NULL,
    brand text NOT NULL,
    displayname text,
    description text,
    calories integer,
    protein numeric,
    sugar numeric,
    fat numeric,
    servingsize text,
    price numeric(6,2),
    mealavailability text[],
    modifiers text[],
    tags text[],
    promptcontext text,
    embedding double precision[],
    currency text,
    updatedon timestamp without time zone NOT NULL,
    categories text[],
    tagsreviewedon timestamp without time zone,
    tagsreviewederror text,
    organizationid text,
    locationid text,
    categoryid text,
    item_class_type integer
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	menuentities_pkey	entityid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts targets this table. Note it is distinct from ml.menu_entities (which IS actively populated by ml.usp_refresh_menu_entities from dim.menuitem/dim.organizationlocation/dim.category_hierarchy) — dim.menuentities appears to be an earlier or parallel dimension design that was superseded by the ml schema version and left in place.

4.25 dim.menuitem

One row per menu item (name, entity type, nutritional attributes, price, catalog) — one of the most heavily joined dimension tables, feeding fact.transactionitem, fact.itemmodifier, fact.location_menu_preferences, fact.customer_menu_preferences and the ml schema.

DDL

```
CREATE TABLE dim.menuitem (
    id bigint DEFAULT nextval('dim.menuitem_id_seq'::regclass) NOT NULL,
    menuitemid text NOT NULL,
    menuitemname text NOT NULL,
    guest integer DEFAULT 1 NOT NULL,
    effective_date date,
    item_class_type integer,
    entitytype text,
    calories text,
    protein numeric,
    sugar numeric,
    fat numeric,
    is_alcoholic boolean,
    is_vegetarian_item boolean,
    is_vegan_item boolean,
    has_allergen boolean,
    is_active boolean,
    is_deleted boolean,
    gms_created_on timestamp without time zone,
    gms_modified_on timestamp without time zone,
    itemunitprice numeric(12,3),
    price_changed_on timestamp without time zone,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    catalogid text,
    average_rating NUMERIC(3,2),
    rating_count INTEGER
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	menuitem_pk	id
UNIQUE	menuitemid_unq	menuitemid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
ix_dim_menuitem_catalogid	btree	catalogid

How It's Populated

dim.usp_refresh_menuitem is called by DimensionMenuItems::'dim_usp_refresh_kiosk' (the activity name is inherited from an earlier copy of the pipeline template — it actually invokes the menu-item refresh, not the kiosk refresh).

It reads stg.dim_menuitem (a full Copy-Activity snapshot of GMS public.item_master), dedups with DISTINCT ON (menuitemid), and performs a NOT EXISTS-guarded insert of new items plus an UPDATE of nutrition/price/active-flag attributes for existing ones.

fact.usp_nge_update_itemssurvey also writes menu-item-name backfills into this table as a side effect of matching survey responses to menu items.

Historically also fed directly by the NewCosmosOrdersMenus / OldCosmosOrdersMenus / gmsMenuItemsToGAS data flows (Cosmos-order-document-based enrichment) and a UpdateItemClassType data flow that patches item_class_type on public.item_master upstream — these run in parallel with the stored-procedure path above. A single index on catalogid supports catalog-scoped lookups.

4.26 dim.modifier

One row per modifier (name, min/max quantity, price, default flag) — child of dim.modifier_group via dim.modifier_group_mapping.

DDL

```
CREATE TABLE dim.modifier (
    modifierid character varying(50) NOT NULL,
    catalogid character varying(50),
    modifiername character varying(255),
    min_quantity integer,
    max_quantity integer,
    allow_quantity_increment boolean,
    increment_step integer,
    calories text NOT NULL,
    calories_text text,
    is_modifier_active boolean NOT NULL,
    is_modifier_deleted boolean NOT NULL,
    modifier_created_on timestamp without time zone,
    modifier_modified_on timestamp without time zone,
    is_modifier_default boolean,
    modifier_default_quantity integer,
    is_invisible boolean,
    classification integer,
    price numeric(12,3),
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    price_changed_on timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	modifierid_unq	modifierid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
ix_dim_modifier_catalogid	btree	catalogid

How It's Populated

dim.usp_refresh_modifier is called by DimensionMenuModifiers::dim_usp_refresh_modifier.

It reads stg.dim_modifier (a full Copy-Activity snapshot of GMS public.item_master modifier rows), dedups with DISTINCT ON (modifierid), and performs a NOT EXISTS-guarded insert plus attribute UPDATE.

Also fed by the gmsModifiersToGAS data flow (a parallel GMS-direct enrichment path). An index on catalogid supports catalog-scoped lookups from the modifier-matching ML procedure.

4.27 dim.modifier_group

One row per modifier group (name, min/max/free selection counts, slider-mode UI flags) — parent of dim.modifier via dim.modifier_group_mapping.

DDL

```
CREATE TABLE dim.modifier_group (
    modifiergroupid character varying(50) NOT NULL,
    modifiergroupname character varying(510) NOT NULL,
    catalogid character varying(50) NOT NULL,
    max_selection integer,
    min_selection integer,
    free_count integer,
    pos_linked_entity_id character varying(50),
    is_active boolean NOT NULL,
    is_deleted boolean NOT NULL,
    created_on timestamp without time zone,
    modified_on timestamp without time zone,
    negative_modifier_behavior integer,
    created_by character varying(255),
    modified_by character varying(255),
    max_aggregate_count integer,
    min_aggregate_count integer,
    increment_step integer,
    slider_mode boolean DEFAULT false NOT NULL,
    slider_mode_modifier boolean DEFAULT false NOT NULL,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	modifier_group_master_pkey	modifiergroupid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
ix_dim_modifiergroup_catalogid	btree	catalogid

How It's Populated

Both dim.usp_refresh_modifiergroup (called by DimensionMenuModifiers::dim_usp_refresh_modifiergroup) and dim.usp_refresh_modifiergroup_modifier_mapping (despite its name, its FROM/target logic here also lands in dim.modifier_group) read stg.dim_modifiergroup, dedup with DISTINCT ON (modifiergroupid), and perform a NOT EXISTS-guarded insert plus attribute UPDATE.

Also fed by the gmsModifiersToGAS data flow. An index on catalogid supports catalog-scoped lookups.

4.28 dim.modifier_group_mapping

Bridge table linking modifier groups to their member modifiers (modifier_mapping_id surrogate key) — resolves the many-to-many between dim.modifier_group and dim.modifier.

DDL

```
CREATE TABLE dim.modifier_group_mapping (
    modifier_mapping_id character varying(50) NOT NULL,
    modifierid character varying(50) NOT NULL,
    modifiergroupid character varying(50) NOT NULL,
    catalogid character varying(50),
    is_mapping_active boolean,
    is_mapping_deleted boolean,
    mapping_created_on timestamp without time zone,
    mapping_modified_on timestamp without time zone,
    is_default boolean,
    default_quantity integer,
    allow_quantity_increment boolean,
    increment_step integer,
    min_quantity integer,
    max_quantity integer,
    calories_text text,
    is_invisible boolean,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	`modfrgrp_modfr_unq`	`modifiergroupid, modifierid`
PRIMARY KEY	`modifier_group_modifier_glue_pkey`	`modifier_mapping_id`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'CopyModifierGroupMapping' Copy Activity in ChildPipTrainingDataML and by the gmsModifiersToGAS data flow, both sourced from GMS public.item_master.

No PL/pgSQL stored procedure targets this table directly in the reviewed source; it is read (not written) by fact.usp_modifier_interaction_analysis and ml.usp_refresh_modifier_interactions when resolving which group a given modifier belongs to.

4.29 dim.occasionsurvey

One row per (organization, survey) — survey name, type, question type, selection type — definitions for the guest-feedback survey program.

DDL

```
CREATE TABLE dim.occasionsurvey (
    surveykey bigint DEFAULT nextval('dim.occasionsurvey_surveykey_seq'::regclass) NOT
NULL,
    organizationid text NOT NULL,
    surveyid text NOT NULL,
    surveyname text,
    surveytype integer,
    question_type integer,
    selection_type integer,
    survey_status integer,
    is_deleted boolean,
    created_on timestamp without time zone,
    modified_on timestamp without time zone,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	survey_pkey	surveykey

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_occasionsurvey is called by OccasionSurveyOrdersFeedback::MaxNgeItemsSurveys.

It reads stg.dim_occasionsurvey (a full Copy-Activity snapshot from the gasSurveyCosmos container), dedups with DISTINCT ON (organizationid, surveyid), and performs a NOT EXISTS-guarded insert plus attribute UPDATE.

Also fed directly by the dimOccasionSurvey data flow (a parallel Cosmos-direct enrichment path within the same pipeline).

4.30 dim.ordertype

One row per (location, kiosk, order type) — order type label (dine-in/takeout/delivery/etc.) — joined by nearly every transaction-level fact table.

DDL

```
CREATE TABLE dim.ordertype (
    id bigint DEFAULT nextval('dim.ordertype_id_seq'::regclass) NOT NULL,
    locationid text NOT NULL,
    kioskid text NOT NULL,
    ordertypeid text NOT NULL,
    ordertypelabel text NOT NULL,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	ordertype_pk	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
order_type_uidx	btree (unique)	locationid, kioskid, ordertypeid

How It's Populated

dim.usp_refresh_ordertype is called from two places: FactsMaster::dimOrderType (a Lookup step that runs before gasTransactionsToAnalyticsDb in the daily Facts chain) and MasterStoredProcedures::dim_usp_refresh_ordertype (a standalone utility pipeline, not wired into the master trigger chain).

It reads stg.silver_transaction_header directly (not a dedicated stg.dim_* staging table) to discover new order-type labels, dedups with DISTINCT ON (locationid, kioskid, ordertypeid), and performs a NOT EXISTS-guarded insert.

Also fed directly by the CosmosOrderTypes data flow, a parallel Cosmos-direct path. A unique index on (locationid, kioskid, ordertypeid) enforces the natural key at the application level (Citus does not enforce it as a true database constraint).

4.31 dim.ordertype_bkp

Structural backup copy of (an earlier, narrower 5-column version of) dim.ordertype.

DDL

```
CREATE TABLE dim.ordertype_bkp (
    id bigint NOT NULL,
    locationid text NOT NULL,
    kioskid text NOT NULL,
    ordertypeid text NOT NULL,
    ordertypelabel text NOT NULL
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	ordertype_bkp_pk	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table — a manual point-in-time snapshot, consistent with the other '_bkp' tables.

4.32 dim.organization

One row per organization (34 columns: name, address, timezone, CEP subscription flags) — the top-level tenant entity above dim.location.

DDL

```
CREATE TABLE dim.organization (
    id character varying(40) NOT NULL,
    name character varying(255) NOT NULL,
    address1 character varying(255),
    address2 character varying(255),
    city character varying(255),
    state character varying(255),
    zipcode character varying(20),
    country character varying(255),
    organizationtype smallint,
    status smallint,
    phonenumber character varying(20),
    email character varying(255),
    isdeleted boolean DEFAULT false,
    createdon timestamp without time zone,
    createdby character varying(255),
    modifiedon timestamp without time zone,
    modifiedby character varying(255),
    active boolean,
    timezone character varying(50),
    coordinates text,
    dayofweek integer,
    hour integer,
    minutes integer,
    roundupforcharity boolean,
    is_ecm_enabled boolean,
    is_cep_enabled boolean,
    is_concessionaire_enabled boolean,
    is_smart_upsells_enabled boolean,
    is_feedback_survey_enabled boolean,
    is_digital_menu_board_enabled boolean,
    is_digital_menu_default_format_enabled boolean,
    cep_subscriptions text,
    sysinserttime timestamp without time zone,
```

```
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	organization_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_refresh_organization is called by CopyGOUViewsToGAS::dim_usp_refresh_organization, which runs after the dimElement/dimView Lookup steps in the GEM_GSH_GOU_Master chain.

It reads stg.dim_organization (a full Copy-Activity snapshot of the public.organization view), LEFT JOINs dim.kioskdetails and stg.dim_cep_subscriptions for enrichment, and performs a single INSERT ... ON CONFLICT (id) DO UPDATE — the only dim.usp_refresh_* procedure that uses a true upsert rather than the insert-new-then-update-changed pattern used elsewhere.

Also fed directly by the CopyGOUViewsToGAS data flow itself (joining public.organization, gasCosmosLocation and gemSubscription) as a parallel enrichment path.

4.33 dim.organizationlocation

One row per (organization, location) — organization/location names and type — the bridge table joining dim.organization to dim.location, read by nearly every dim.usp_refresh_* and fact.usp_* procedure for location scoping.

DDL

```
CREATE TABLE dim.organizationlocation (
    organizationid character varying(40) NOT NULL,
    organizationname character varying(255),
    locationid character varying(40) NOT NULL,
    locationname character varying(255) NOT NULL,
    organizationtype smallint,
    roundupforcharity boolean,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	organizationid_locationid_pk	organizationid, locationid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
IX_organizationid_locationid	btree	organizationid, locationid
idx_organizationlocation_locationid	btree	locationid
idx_organizationlocation_organizationid	btree	organizationid

How It's Populated

dim.usp_refresh_organizationlocation is called by DimensionOrganizationLocationCatalog::dim_usp_refresh_organizationlocation.

It reads stg.dim_organizationlocation (a full Copy-Activity snapshot of the public.vw_organizationlocation view), joins dim.organization for enrichment, dedups with DISTINCT ON (organizationid, locationid), and performs a NOT EXISTS-guarded insert plus attribute UPDATE — and, as noted above, also backfills dim.location in the same procedure.

Historically also fed by the CopyGOUViewToGASold data flow (legacy path). Three indexes — a composite (organizationid, locationid) with an INCLUDE of organizationname/locationname, plus single-column locationid and organizationid — support the very high fan-out of joins against this table.

4.34 dim.peripheral

One row per device peripheral (printer, scanner, card reader, etc.) attached to a kiosk — device-health companion to dim.device.

DDL

```
CREATE TABLE dim.peripheral (
    id bigint NOT NULL,
    deviceid bigint,
    peripheralid character varying(50) NOT NULL,
    peripheraltype character varying(50) NOT NULL,
    state character varying(50) NOT NULL,
    previousstate character varying(50),
    statechangedate timestamp without time zone,
    description text,
    model text,
    serial text,
    ipaddress character varying(50)
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	peripheral_deviceid_peripheralid_key	deviceid, peripheralid
PRIMARY KEY	peripheral_pkey	id
FOREIGN KEY	peripheral_deviceid_fkey	deviceid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
peripheral_idx	btree	deviceid, peripheralid

How It's Populated

Populated directly by the DevicesAndPeripherals and gshDevicesAndPeripheralsToAnalyticsDb data flows, sourced from gsh.peripheral (joined with gsh.device) in the GSH device-health platform.

No PL/pgSQL procedure targets this table directly. An index on (deviceid, peripheralid) supports the natural-key lookup used by the data flow's upsert logic.

4.35 dim.upsellgrouplookup

One row per upsell group (name/category of upsell offer) — joined by fact.usp_offer_analysis when building fact.vw_offer_analysis.

DDL

```
CREATE TABLE dim.upsellgrouplookup (
    upsellgroupid character varying(50) NOT NULL,
    upsellgroupname text,
    isactive boolean,
    createdon timestamp without time zone,
    modifiedon timestamp without time zone,
    catalogid text COLLATE pg_catalog."default",
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	upsellgroupid_pkey	upsellgroupid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'dimUpsellGroupLookup' Copy Activity in UpsellRecommendationsToGAS, sourced from public.upsell_group — a full snapshot copy on every run.

The UpdateUpsellRecommendations data flow separately reads fact.vw_offer_analysis joined to public.upsell_group to refresh derived attributes. No PL/pgSQL procedure targets this table directly.

4.36 dim.userlocation

Two-column (userid, locationid) association table recording which back-office users have access to which locations.

DDL

```
CREATE TABLE dim.userlocation (
    userid character varying(40) NOT NULL,
    locationid character varying(40) NOT NULL,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`userlocation_pkey`	`userid, locationid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the 'dimUserLocation' Copy Activity in DimensionOrganizationLocationCatalog, sourced from the public.vw_userlocation view — a full snapshot copy on every run. No PL/pgSQL procedure targets this table.

4.37 dim.view

Small lookup of GEM UI 'view'/screen identifiers referenced by fact.userbehaviour for clickstream/navigation analysis — the screen-level counterpart to dim.element.

DDL

```
CREATE TABLE dim.view (
    viewid integer DEFAULT nextval('dim.view_id_seq'::regclass),
    viewname text,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
`idx_view_viewid`	btree	`viewid`

How It's Populated

dim.usp_refresh_view is called by GEM_GSH_GOU_Master::dimView, immediately after dimElement in that chain (dimElement -> dimView -> CopyGOUViewsToGAS).

It reads distinct new view values out of stg.silver_kiosk_events (NOT EXISTS-guarded against dim.view) and inserts them — the same simple append-only discovery pattern as dim.element.

Also referenced (read-only) by the gemUserBehaviourEventsToFact family of data flows when building fact.userbehaviour. An index on viewid supports the lookup.

4.38 dim.vw_grubbrrinstallbase

Wide (142-column) reporting table — despite the 'vw_' naming it is a physical TABLE, not a database view — that flattens kiosk, install-base and organization/location attributes into one row per (location, kiosk) for the Grubbrr install-base Power BI dashboard.

DDL

```
CREATE TABLE dim.vw_grubbrrinstallbase (
    organization_id character varying(50) NOT NULL,
    organization_name text NOT NULL,
    location_id character varying(50) NOT NULL,
    location_name text NOT NULL,
    kiosk_id character varying(50) NOT NULL,
    kiosk_name text,
    kiosk_hardware_id character varying(50),
    kiosk_software_version character varying(50),
    os_type character varying(50),
    serial_number character varying(50),
    is_test_mode boolean,
    is_demo_kiosk boolean,
    is_test_mode_on boolean,
    last_login_time timestamp without time zone,
    last_sync_time timestamp without time zone,
    device_created_on timestamp without time zone,
    device_deleted_on timestamp without time zone,
    is_test_kiosk boolean,
    device_type character varying(50),
    is_activated boolean,
    payment_integration_configs jsonb,
    printer_configs jsonb,
    kiosk_activation character varying(50),
    is_kiosk_deleted boolean,
    kiosk_mode character varying(10),
    kiosk_logging integer,
    is_goast_kiosk boolean,
    loyalty_login_otp character varying(50),
    pos_provider jsonb,
    payment_provider jsonb,
    payment_device_type character varying(50),
    loyalty_provider jsonb,
    scanners jsonb,
    organization_status character varying(20),
    location_status character varying(20),
    is_org_active boolean,
    is_loc_active boolean,
    is_org_deleted boolean,
    is_loc_deleted boolean,
```

```
    org_go_live_date timestamp without time zone,
    loc_go_live_date timestamp without time zone,
    org_created_date timestamp without time zone,
    loc_created_date timestamp without time zone,
    is_org_ecm_enabled boolean,
    is_org_cep_enabled boolean,
    is_org_concessionaire_enabled boolean,
    is_org_smart_upsells_enabled boolean,
    is_org_feedback_survey_enabled boolean,
    is_org_digital_menu_board_enabled boolean,
    is_org_digital_menu_default_format_enabled boolean,
    is_loc_ecm_enabled boolean,
    is_loc_cep_enabled boolean,
    is_loc_concessionaire_enabled boolean,
    is_loc_smart_upsells_enabled boolean,
    is_loc_feedback_survey_enabled boolean,
    is_loc_digital_menu_board_enabled boolean,
    is_loc_digital_menu_default_format_enabled boolean,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    item_special_request jsonb,
    legal_copy_enabled boolean,
    ada_configuration jsonb,
    calculate_default_modifier_price boolean,
    track_kiosk_user_behavior boolean,
    loyalty_feature boolean,
    pickup_flow boolean,
    pos_auto_applied_discount boolean,
    search_functionality_enabled boolean,
    recent_orders_enabled boolean,
    play_card_config jsonb,
    round_up_for_charity boolean,
    calories_enabled boolean,
    scan_and_go_enabled boolean,
    age_verification jsonb,
    tips_enabled boolean,
    apply_before_taxes boolean,
    auto_print_enabled boolean,
    include_pos_order_number boolean,
    show_qr_code_when_print_receipt_fails boolean,
    print_modifier_group_names boolean,
    print_default_modifiers boolean,
    print_free_modifiers boolean,
    print_priced_modifiers boolean,
    enable_email_receipts boolean,
    enable_sms_receipt boolean,
    qr_code_for_receipt boolean,
    show_screensaver boolean,
    business_hours_show_message boolean,
    business_hours_enabled boolean,
    pos_hours_enabled boolean,
    quantity_limit_per_item integer,
    quantity_limit_per_order integer,
    max_discount_per_order integer,
    show_item_asis_option boolean,
    enable_minimum_order_total boolean,
    auto_apply_min_qty_to_first_modifier boolean,
    show_make_it_a_meal_option boolean,
    enable_combo_auto_skip boolean,
    number_of_item_upsell_prompts_per_order integer,
    can_enter_code_for_discount boolean,
    can_scan_qr_code_for_discount boolean,
    can_select_from_list_for_discount boolean,
```

```
    enabled_languages jsonb,
    display_modifier_group_restriction boolean,
    allow_user_to_collapse_or_expand_modifier_groups boolean,
    show_modifier_group_names_on_order_review boolean,
    show_default_modifier_on_order_review boolean,
    auto_expand_modifier_group boolean,
    enable_nested_modifier_indentation boolean,
    open_nested_modifiers_in_popup boolean,
    category_header_display_mode text,
    category_header_logo_display_mode text,
    show_item_description boolean,
    category_name_position text,
    hide_sold_out_item_and_modifier_on_kiosk boolean,
    make_category_sidebar_translucent boolean,
    enable_single_step_subcategory_flow boolean,
    remove_category_highlighted_border boolean,
    enable_extended_combo_mode boolean,
    button_style text,
    show_discount_code_button boolean,
    make_item_combo_images_rounded boolean,
    show_loyalty_points_on_header boolean,
    show_card_accepted_payment_options boolean,
    show_google_pay_accepted_payment_options boolean,
    show_apple_pay_accepted_payment_options boolean,
    show_cash_accepted_payment_options boolean,
    show_tap_to_order_cta boolean,
    use_text_for_cta boolean,
    use_image_for_cta boolean,
    choose_a_currency jsonb,
    choose_a_locale text,
    order_number_start integer,
    allotment integer,
    negative_modifier_behavior jsonb,
    disclaimer_text text,
    preorder_popup_enabled boolean,
    preorder_popup_text text,
    order_types_identity_config jsonb,
    show_category_highlighted_color boolean,
    cep_subscriptions jsonb,
    perform_pos_status_check boolean
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`locationid_deviceid_pk`	`location_id, kiosk_id`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_grubbrr_install_base is called by GrubbrrInstallBaseDashboard::dim_usp_grubbrr_install_base on the TwiceDaily trigger.

It TRUNCATEs this table and rebuilds it in full each run from dim.kioskdetails and dim.organizationlocation, joined to dim.kiosk and dim.organization — a full-refresh reporting extract rather than an incremental table, appropriate given it directly backs a dashboard.

4.39 dim.vw_grubbrrinstallbase_all_devices

Same shape and purpose as dim.vw_grubbrrinstallbase, but scoped to all devices at a location (not just the primary kiosk) — used where the dashboard needs a device-level rather than kiosk-level breakdown.

DDL

```
CREATE TABLE dim.vw_grubbrrinstallbase_all_devices (
    organization_id character varying(50) NOT NULL,
    organization_name text NOT NULL,
    location_id character varying(50) NOT NULL,
    location_name text NOT NULL,
    kiosk_id character varying(50) NOT NULL,
    kiosk_name text,
    kiosk_hardware_id character varying(50),
    kiosk_software_version character varying(50),
    os_type character varying(50),
    serial_number character varying(50),
    is_test_mode boolean,
    is_demo_kiosk boolean,
    is_test_mode_on boolean,
    last_login_time timestamp without time zone,
    last_sync_time timestamp without time zone,
    device_created_on timestamp without time zone,
    device_deleted_on timestamp without time zone,
    is_test_kiosk boolean,
    device_type character varying(50),
    is_activated boolean,
    payment_integration_configs jsonb,
    printer_configs jsonb,
    kiosk_activation character varying(50),
    is_kiosk_deleted boolean,
    kiosk_mode character varying(10),
    kiosk_logging integer,
    is_goast_kiosk boolean,
    loyalty_login_otp character varying(50),
    pos_provider jsonb,
    payment_provider jsonb,
    payment_device_type character varying(50),
    loyalty_provider jsonb,
    scanners jsonb,
    organization_status character varying(20),
    location_status character varying(20),
    is_org_active boolean,
    is_loc_active boolean,
    is_org_deleted boolean,
    is_loc_deleted boolean,
    org_go_live_date timestamp without time zone,
    loc_go_live_date timestamp without time zone,
    org_created_date timestamp without time zone,
    loc_created_date timestamp without time zone,
    is_org_ecm_enabled boolean,
    is_org_cep_enabled boolean,
    is_org_concessionaire_enabled boolean,
    is_org_smart_upsells_enabled boolean,
    is_org_feedback_survey_enabled boolean,
```

```
    is_org_digital_menu_board_enabled boolean,
    is_org_digital_menu_default_format_enabled boolean,
    is_loc_ecm_enabled boolean,
    is_loc_cep_enabled boolean,
    is_loc_concessionaire_enabled boolean,
    is_loc_smart_upsells_enabled boolean,
    is_loc_feedback_survey_enabled boolean,
    is_loc_digital_menu_board_enabled boolean,
    is_loc_digital_menu_default_format_enabled boolean,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    item_special_request jsonb,
    legal_copy_enabled boolean,
    ada_configuration jsonb,
    calculate_default_modifier_price boolean,
    track_kiosk_user_behavior boolean,
    loyalty_feature boolean,
    pickup_flow boolean,
    pos_auto_applied_discount boolean,
    search_functionality_enabled boolean,
    recent_orders_enabled boolean,
    play_card_config jsonb,
    round_up_for_charity boolean,
    calories_enabled boolean,
    scan_and_go_enabled boolean,
    age_verification jsonb,
    tips_enabled boolean,
    apply_before_taxes boolean,
    auto_print_enabled boolean,
    include_pos_order_number boolean,
    show_qr_code_when_print_receipt_fails boolean,
    print_modifier_group_names boolean,
    print_default_modifiers boolean,
    print_free_modifiers boolean,
    print_priced_modifiers boolean,
    enable_email_receipts boolean,
    enable_sms_receipt boolean,
    qr_code_for_receipt boolean,
    show_screensaver boolean,
    business_hours_show_message boolean,
    business_hours_enabled boolean,
    pos_hours_enabled boolean,
    quantity_limit_per_item integer,
    quantity_limit_per_order integer,
    max_discount_per_order integer,
    show_item_asis_option boolean,
    enable_minimum_order_total boolean,
    auto_apply_min_qty_to_first_modifier boolean,
    show_make_it_a_meal_option boolean,
    enable_combo_auto_skip boolean,
    number_of_item_upsell_prompts_per_order integer,
    can_enter_code_for_discount boolean,
    can_scan_qr_code_for_discount boolean,
    can_select_from_list_for_discount boolean,
    enabled_languages jsonb,
    display_modifier_group_restriction boolean,
    allow_user_to_collapse_or_expand_modifier_groups boolean,
    show_modifier_group_names_on_order_review boolean,
    show_default_modifier_on_order_review boolean,
    auto_expand_modifier_group boolean,
    enable_nested_modifier_indentation boolean,
    open_nested_modifiers_in_popup boolean,
    category_header_display_mode text,
```

```
    category_header_logo_display_mode text,
    show_item_description boolean,
    category_name_position text,
    hide_sold_out_item_and_modifier_on_kiosk boolean,
    make_category_sidebar_translucent boolean,
    enable_single_step_subcategory_flow boolean,
    remove_category_highlighted_border boolean,
    enable_extended_combo_mode boolean,
    button_style text,
    show_discount_code_button boolean,
    make_item_combo_images_rounded boolean,
    show_loyalty_points_on_header boolean,
    show_card_accepted_payment_options boolean,
    show_google_pay_accepted_payment_options boolean,
    show_apple_pay_accepted_payment_options boolean,
    show_cash_accepted_payment_options boolean,
    show_tap_to_order_cta boolean,
    use_text_for_cta boolean,
    use_image_for_cta boolean,
    choose_a_currency jsonb,
    choose_a_locale text,
    order_number_start integer,
    allotment integer,
    negative_modifier_behavior jsonb,
    disclaimer_text text,
    preorder_popup_enabled boolean,
    preorder_popup_text text,
    order_types_identity_config jsonb,
    show_category_highlighted_color boolean,
    cep_subscriptions jsonb,
    perform_pos_status_check boolean
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`locationid_kioskid_pk`	`location_id, kiosk_id`
FOREIGN KEY	`organizationid_locationid_fk`	`organization_id, location_id`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

dim.usp_grubbrr_install_base_all_devices is defined in the stored-procedure source but is not called by name from any Lookup activity's query text across the 67 reviewed pipelines — it is presumably invoked manually or from a pipeline/tool not captured in this ARM template export.

Its logic mirrors dim.usp_grubbrr_install_base: TRUNCATE-and-rebuild from dim.kioskdetails and dim.organizationlocation joined to dim.kiosk and dim.organization.

4.40 dim.weather

One row per (location, apicalldate) storing a full day's raw hourly weather payload as JSONB (weatherinfo) plus location metadata (city, timezone) — unpacked by dim.vw_weatherhourlydata into per-hour rows for ml.usp_refresh_weather.

DDL

```
CREATE TABLE dim.weather (
    organizationid text,
    locationid text NOT NULL,
    city text,
    timezone text,
    apicalldate date NOT NULL,
    locationinfo jsonb,
    weatherinfo jsonb,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`location_date_pk`	`locationid, apicalldate`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated by the DimensionWeatherInfo pipeline (not part of the four scheduled master triggers — appears to be run manually or on a separate/disabled schedule): 'InsertLocations' first inserts one placeholder row per (location, target apicalldate) selected from dim.location, filtered to only locations without an existing row for that date.

'LoopThroughCities' then iterates the distinct cities (currently filtered to a Florida city list in the Lookup query reviewed: West Palm Beach, Tampa, Fort Lauderdale) and, per city, calls the WeatherAPI.com history endpoint via a WebActivity and immediately UPDATEs the placeholder row's weatherinfo/locationinfo/timezone JSONB columns with the API response.

This is a REST-API-driven enrichment pattern rather than a database Copy Activity or stored procedure, and — per the trigger list — is not currently on the same daily schedule as the rest of the warehouse, so its freshness should be verified independently.

4.41 dim.weather_bkp

Structural backup copy of (an earlier shape of) dim.weather, keyed by (locationid, businessdate) rather than apicalldate.

DDL

```
CREATE TABLE dim.weather_bkp (
    organizationid text,
    locationid text NOT NULL,
    city text,
    timezone text,
    businessdate date NOT NULL,
    weatherinfo jsonb,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	location_date_bkp_pk	locationid, businessdate

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table — a manual point-in-time snapshot, consistent with the other '_bkp' tables, taken before a rekey/redesign of the weather table (apicalldate replacing businessdate as the date key).

5. FACT Schema - Fact Tables

Thirty-three fact tables covering orders, device health, clickstream behavior, surveys, recommendations and pipeline operations.

5.1 fact.cep_incidents

One row per GEM Connector/CEP incident (connectivity, hardware, ordering-flow error) reported by a kiosk — a device-health/support fact table.

DDL

```
CREATE TABLE fact.cep_incidents (
    incidentkey bigint,
    application text,
    organizationid text,
    locationid text,
    deviceid text,
    eventmodule text,
    eventcategory text,
    eventtype text,
    eventtoken text,
    incidenttype text,
    incidentcount integer,
    eventinstant text,
```

```
    firstoccurred timestamp without time zone,
    lastoccurred timestamp without time zone,
    notificationtypeid text,
    incidentdata text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    severity text
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_silver_cep_incidents_to_fact_cep_incidents (called by gemEventsToAnalyticsDb::factCEPIncidents, right after factDeviceEvent in the GEM_GSH_GOU_Master chain) reads new rows from stg.silver_cep_incidents past the fact.watermarktable checkpoint, dedups with DISTINCT ON (id, token), joins dim.organizationlocation and fact.gem_failed_order_job_notifications for enrichment, and uses a NOT EXISTS anti-join before inserting.

Also fed directly by the EventsFromCosmosGEMToFact / MergedEventsFromCosmosGEMToFact / gemCepIncidents data flows, which build the same fact table straight from Cosmos DB gemEvent/gemJobs documents as a parallel/legacy path.

5.2 fact.customer_menu_preferences

One row per (organization, location, frequent customer, day-part, item) capturing which items a loyalty customer tends to order at which time of day — used for personalization features.

DDL

```
CREATE TABLE fact.customer_menu_preferences (
    organizationid character varying(50) NOT NULL,
    locationid character varying(50) NOT NULL,
    frequentcustomerid character varying(50) NOT NULL,
    day_parts character varying(20) NOT NULL,
    itemid character varying(50),
    itemtype character varying(50),
    item_selection_frequency integer,
    itemtags jsonb,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_customer_menu_preferences (called by GrubbrrInstallBaseDashboard::dim_usp_grubbrr_install_base, i.e. it rides along on the twice-daily install-base refresh rather than the main Facts chain) TRUNCATEs this table and rebuilds it in full from fact.transactionheader joined to fact.transactionitem and dim.frequentcustomer, using a ROW_NUMBER()-based aggregation to rank each customer's most-ordered items per day-part.

5.3 fact.deviceevent

One row per kiosk device event (data category, action type, event token/instant) — the general-purpose device event stream, narrower and more operational than fact.userbehaviour's clickstream focus.

DDL

```
CREATE TABLE fact.deviceevent (
    application text NOT NULL,
    companyid text NOT NULL,
    locationid text NOT NULL,
    moduleid text,
    datacategory text,
    actiontype text,
    severity text,
    eventtoken text,
    eventinstant text,
    dateid integer,
    username text,
    userid text,
    deviceid text,
    devicename text,
    summary text,
    eventdata text,
    syscosmosticks bigint,
    sysinserttime timestamp without time zone,
    syscosmosts bigint,
    sysupdatetime TIMESTAMP
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
FOREIGN KEY	orgid_locationid_fk	companyid, locationid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
ix_deviceevent_journey_lookup	btree	locationid, dateid, datacategory, eventtoken

How It's Populated

fact.usp_silver_kiosk_events_to_fact_deviceevent (called by gemEventsToAnalyticsDb::factDeviceEvent, the first step in that pipeline) reads new rows from stg.silver_kiosk_events past the fact.watermarktable checkpoint, dedups with a

COALESCE(NULLIF(token,...)) DISTINCT ON expression, joins dim.organizationlocation, and inserts with ON CONFLICT (locationid, eventtoken, datacategory, actiontype, eventinstant) DO NOTHING (a pure dedup-guard, not an upsert).

Also fed directly by the EventsFromCosmosGEMToFact / MergedEventsFromCosmosGEMToFact data flows (parallel/legacy Cosmos-direct path). A composite index on (locationid, dateid, datacategory, eventtoken) supports the common 'event journey' lookup pattern (all events for one order token on one day).

Has a (currently disabled, Citus-incompatible) logical foreign key on (companyid, locationid) pointing at the company/location dimensions, documenting the intended relationship even though it isn't enforced by the engine.

5.4 fact.devicehealth

One row per device-health snapshot event mirrored from the GSH platform's own devicehealth table — a lighter-weight companion to stg.fact_devicestate / fact.devicestate that stores the health payload with its own surrogate key.

DDL

```
CREATE TABLE fact.devicehealth (
    id bigint NOT NULL,
    healthdatatype character varying(50) NOT NULL,
    locationid character varying(50) NOT NULL,
    companyid character varying(50) NOT NULL,
    deviceid character varying(50) NOT NULL,
    devicetype character varying(50) NOT NULL,
    status character varying(50) NOT NULL,
    statusmessage text,
    healthdatatime timestamp without time zone NOT NULL,
    statuschangetime timestamp without time zone NOT NULL,
    inserttime timestamp without time zone NOT NULL,
    version character varying(50),
    devicedatatime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	devicehealth_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated directly by the gshDeviceHealthToAnalyticsDb and gshDevicesAndPeripheralsToAnalyticsDb data flows, both sourced from gsh.devicehealth. No PL/pgSQL procedure targets this table directly in the reviewed source — dim.usp_grubbrr_install_base_all_devices), which is not itself called from any pipeline, is the closest structural relative.

5.5 fact.devicestate

One row per (location, device) health-state snapshot at a point in time (dateid-scoped) — the primary device-health fact table used for kiosk uptime/incident reporting.

DDL

```
CREATE TABLE fact.devicestate (
    id bigint DEFAULT nextval('fact.devicestate_id_seq'::regclass) NOT NULL,
    companyid TEXT,
    locationid TEXT,
    deviceid TEXT,
    dateid INTEGER,
    state TEXT,
    lasteventtime TIMESTAMP,
    statuschangetime TIMESTAMP,
    duration NUMERIC(10,3),
    sysinserttime TIMESTAMP,
    status TEXT,
    --added on 2026-06-19 for enhanced System Health Report statusmessage TEXT,
    --added on 2026-06-19 for enhanced System Health Report healthdatatype TEXT,
    --added on 2026-06-19 for enhanced System Health Report sysupdatetime TIMESTAMP,
    CONSTRAINT locationid_deviceid_lasteventtime_pkey PRIMARY KEY (locationid, deviceid,
lasteventtime)
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_devicestate	btree	locationid, companyid, deviceid
idx_devicestate_dateid	btree	dateid
idx_devicestate_locationid	btree	locationid

How It's Populated

fact.usp_gsh_devicehealth_to_fact_devicestate (called by gshDeviceHealthToAnalyticsDbFactDeviceState::uspGshDeviceStateToFact, the last step of the GEM_GSH_GOU_Master sub-chain before gshDeviceTelemetryToAnalyticsDb) reads new rows from stg.fact_devicestate past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, deviceid, healthdatatime), joins dim.kiosk and dim.organization, and inserts (plain INSERT, no ON CONFLICT — relies on the anti-join for idempotency).

Also fed directly by the TestDeviceHealth and gshDeviceHealthToAnalyticsDbDeviceState data flows (a full-load/backfill variant and the primary data-flow-based path respectively), both of which also update fact.watermarktable themselves.

Three indexes — composite (locationid, companyid, deviceid), plus single-column dateid and locationid — support the dashboard-style rollups run against this table.

5.6 fact.devicetelemetry

One row per (location, device, date) telemetry/heartbeat summary from the GSH platform — the aggregate counterpart to the more granular fact.devicestate snapshots.

DDL

```
CREATE TABLE fact.devicetelemetry (
    deviceid text NOT NULL,
    locationid text NOT NULL,
    dateid integer NOT NULL,
    cpuvalue numeric(10,5),
    memoryvalue numeric(10,5),
    cputimestamp timestamp without time zone,
    memorytimestamp timestamp without time zone,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    CONSTRAINT devicetelemetry_pkey PRIMARY KEY (locationid, deviceid, dateid)
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	devicetelemetry_pkey	locationid, deviceid, dateid
FOREIGN KEY	location_deviceid_fk	locationid, deviceid
FOREIGN KEY	locationid_fk	locationid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
idx_devicetelemetry_dateid	btree	dateid
idx_devicetelemetry_locationid	btree	locationid

How It's Populated

fact.usp_gsh_devicetelemetry_to_fact_devicetelemetry (called by gshDeviceTelemetryToAnalyticsDb::stgDeviceTelemetryToFact) reads new rows from stg.fact_devicetelemetry past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, deviceid, dateid), joins dim.kiosk and dim.organization, and performs an INSERT ... ON CONFLICT (locationid, deviceid, dateid) DO UPDATE.

Also fed directly by the UpdateGshDeviceTelemetryToAnalyticsDb, gshDeviceTelemetryToAnalyticsDb and gshDeviceTelemetryToAnalyticsDbOld data flows (current and legacy variants of the same enrichment). Two indexes on dateid and locationid support reporting rollups.

Carries (currently disabled/Citus-incompatible) logical foreign keys on (locationid, deviceid) and locationid alone, documenting its intended relationship to dim.device/dim.location.

5.7 fact.gem_failed_order_job_notifications

One row per failed background-job notification from the GEM platform (e.g. an order-sync or webhook failure) — an operational/support fact table, also used to enrich fact.cep_incidents.

DDL

```
CREATE TABLE fact.gem_failed_order_job_notifications (
    incidentid BIGINT,
    application TEXT COLLATE pg_catalog."default",
    organizationid TEXT COLLATE pg_catalog."default",
    locationid TEXT COLLATE pg_catalog."default",
    eventmodule TEXT COLLATE pg_catalog."default",
    eventcategory TEXT COLLATE pg_catalog."default",
    eventtype TEXT COLLATE pg_catalog."default",
    eventtoken TEXT COLLATE pg_catalog."default",
    incidentcount INTEGER,
    firstoccurred TEXT COLLATE pg_catalog."default",
    lastoccurred TEXT COLLATE pg_catalog."default",
    incidenttype TEXT COLLATE pg_catalog."default",
    notificationtypeid TEXT COLLATE pg_catalog."default",
    syscosmosts BIGINT,
    sysinserttime TIMESTAMP WITHOUT TIME ZONE
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_stg_gem_failed_order_job_notifications_to_fact (called by gemEventsToAnalyticsDb::factCEPIncidents — the same activity step also drives the CEP-incidents procedure that reads from this table) reads new rows from stg.gem_failed_order_job_notifications past the fact.watermarktable checkpoint, dedups with DISTINCT ON (incidentid, eventtoken), and inserts new failures.

5.8 fact.itemmodifier

One row per (order, item, modifier group, modifier) selection — the modifier-selection detail behind every line item in fact.transactionitem, and the base table for the modifier impression/interaction/recommendation analysis procedures.

DDL

```
CREATE TABLE fact.itemmodifier (
    transactionheaderid text NOT NULL,
    orderid text NOT NULL,
    itemid text NOT NULL,
    modifiergroupid text NOT NULL,
    modifierid text NOT NULL,
    modifiername text,
    modifierquantity smallint DEFAULT 1 NOT NULL,
    modifierprice numeric(12,3),
```

```
    freequantity integer,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    locationid text,
    businessdate date,
    syscosmosts bigint
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	trxnid_itemid_mdfrgrpid_mdfrid_pk	transactionheaderid, itemid, modifiergroupid, modifierid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
idx_fact_itemmodifier_locationid	btree	locationid
ix_itemmodifier_syscosmosts_brin	brin	syscosmosts

How It's Populated

fact.usp_silver_item_modifiers_to_fact (called by gasTransactionsToAnalyticsDb::factItemModifiers, immediately after factTrxnItem in the daily orders chain) reads new rows from stg.silver_item_modifiers past the fact.watermarktable checkpoint, dedups with DISTINCT ON (transactionheaderid, orderitemid, options_modifiergroupid, options_modifierid), joins fact.transactionitem for linkage, and performs an INSERT ... ON CONFLICT (transactionheaderid, itemid, modifiergroupid, modifierid) DO UPDATE.

Also fed directly by the GXSgasOrderItemsFromCosmosToAnalyticsDb / UpdateItemModifier / UpdateItemsAndPaymentsWithHighAmounts / gasOrderItemsFromCosmosToAnalyticsDb data flows — a family of Cosmos-direct enrichment paths (including a specific high-order-amount correction variant) that run in parallel with the stored-procedure path.

A BRIN index on syscosmosts and a b-tree on locationid support the very high row-volume incremental scans and location-scoped reporting queries respectively.

5.9 fact.itemssurvey

One row per (order, item, survey) guest-feedback response about a specific menu item — the item-level counterpart to fact.occasionsurveydetail's order-level feedback.

DDL

```
CREATE TABLE fact.itemssurvey (
    organizationid text,
    locationid text,
    dateid integer,
    surveyid text,
```

```
    surveytransid text,
    orderid text,
    itemid text,
    itemrating text,
    surveytransstatus text,
    surveyissuedtimestamp text,
    surveycompletedtimestamp text,
    surveylocaltimestamp timestamp without time zone,
    sysinserttime timestamp without time zone,
    nge_syscosmosts bigint,
    ordersessionid text,
    gem_event_category text,
    gem_event_type text,
    is_responded boolean,
    gem_syscosmosts bigint,
    gem_event_instant text,
    sysupdatetime timestamp without time zone,
    sourceid integer
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
FOREIGN KEY	`locationid_transactionheaderid_fk`	`locationid, orderid`
FOREIGN KEY	`orgid_locationid_fk`	`organizationid, locationid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_nge_update_itemssurvey (called by OccasionSurveyOrdersFeedback::UpdateRespondedSurveys) reads new rows from stg.fact_itemssurvey past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, orderid, itemid, surveyid, surveytransid), joins fact.transactionheader, dim.organizationlocation and dim.occasionsurvey, and inserts (also backfilling dim.menuitem names as a side effect).

fact.usp_sent_surveys_to_fact_itemssurvey (called by OccasionSurveyOrdersFeedback::SentSurveysToFactItemsSurvey) is a second, independent contributor: it reads fact.sent_surveys joined to fact.transactionheader, dedups with DISTINCT ON (locationid, transactionheaderid, itemid, surveyid), and inserts survey-sent (not-yet-responded) placeholder rows using a double NOT EXISTS guard.

fact.usp_update_occasion_survey_datetime_fields subsequently backfills business-local date/time fields on this table by joining dim.organizationlocation/dim.location/dim.organization.

Also fed directly by the factItemsSurvey data flow. Carries (disabled/Citus-incompatible) logical foreign keys on (locationid, orderid) and (organizationid, locationid).

5.10 fact.location_menu_preferences

One row per (location, day-part, item, item-type) capturing which items are popular at which location and time of day — the location-level counterpart to fact.customer_menu_preferences.

DDL

```
CREATE TABLE fact.location_menu_preferences (
    organizationid character varying(50) NOT NULL,
    locationid character varying(50) NOT NULL,
    day_parts character varying(20) NOT NULL,
    itemid character varying(50),
    itemtype character varying(50),
    item_selection_frequency integer,
    itemtags jsonb,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	location_dayparts_itemid_itemtype_unq	locationid, day_parts, itemid, itemtype

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_location_menu_preferences (called by GrubbrrInstallBaseDashboard::dim_usp_grubbrr_install_base, riding along on the twice-daily install-base refresh) TRUNCATEs and rebuilds this table in full from fact.transactionheader joined to fact.transactionitem, dim.organizationlocation and dim.frequentcustomer, using a ROW_NUMBER()-based ranking per location/day-part.

A unique constraint on (locationid, day_parts, itemid, itemtype) documents the intended one-row-per-combination grain.

5.11 fact.location_statistics

One row per location with rolled-up operational statistics (order counts, item counts, kiosk counts, etc.) — a wide (28-column) summary table feeding dashboards and ML feature preparation.

DDL

```
CREATE TABLE fact.location_statistics (
    organizationid character varying(50),
    organizationname character varying(255),
    locationid character varying(50),
    locationname character varying(255),
    city character varying(255),
    state character varying(255),
    country character varying(255),
```

```
    isactive boolean,
    timezone character varying(255),
    order_type_labels jsonb,
    loc_item_popularity jsonb,
    loc_total_order_count integer,
    loc_total_sales_amount numeric(12,3),
    loc_avg_order_amount numeric(12,3),
    org_total_order_count integer,
    org_total_sales_amount numeric(12,3),
    org_avg_order_amount numeric(12,3),
    number_of_frequent_customers integer,
    orders_placed_by_freq_customers integer,
    amount_spent_by_freq_customers numeric(12,3),
    avg_amount_spent_by_freq_customers numeric(12,3),
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_location_statistics (called by both ChildPipTrainingDataML::GetLocationAndItemStatistics and RefreshMLTables::GetLocationAndItemStatistics, i.e. it runs as part of the daily ML-tables refresh) TRUNCATEs this table and rebuilds it in full from dim.organizationlocation, dim.frequentcustomer, fact.transactionheader, ml.transactions and dim.kioskdetails, joined to dim.menuitem and dim.organization.

5.12 fact.modifier_impressions

One row per (order, item, modifier) impression — records that a modifier was shown/offered to the guest during ordering, whether or not it was selected. Feeds the impression-to-interaction-to-recommendation funnel analysis.

DDL

```
CREATE TABLE fact.modifier_impressions (
    locationid text NOT NULL,
    transactionheaderid text NOT NULL,
    ordersessionid text,
    orderid text,
    menuitemid text,
    modifierid text NOT NULL,
    parent_modifier_id text,
    selection_type text,
    nesting_depth integer,
    "position" integer,
    score numeric(5,3),
    strategy text,
    context text,
    selected boolean,
    pre_deselected boolean,
    confirmed_removed boolean,
    pre_selected boolean,
```

```
    businessdate date,
    orderdatelocal timestamp without time zone,
    frequentcustomerid text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_silver_modifier_impressions_to_fact (called by ModifierUpsellRecommendationsToGAS::factModfrImpressions) reads new rows from stg.silver_modifier_impressions past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, transactionheaderid, menuitemid, modifierid, position), joins dim.organization, and inserts via a NOT EXISTS anti-join.

fact.usp_modifier_impression_analysis (called by the same pipeline's StgToFact step) is a second contributor that derives additional impression rows from fact.modifier_recommendations.

fact.usp_modifier_recommendation_analysis (defined but not called by name from any reviewed Lookup query — likely superseded by the two procedures above, or invoked from a pipeline/tool outside this ARM export) also targets this table, joining fact.transactionitem, fact.itemmodifier and dim.modifier.

5.13 fact.modifier_interactions

One row per (order, modifier) guest-interaction event (tap/select/deselect on a recommended modifier) — the behavioral signal that feeds ml.usp_refresh_modifier_interactions.

DDL

```
CREATE TABLE fact.modifier_interactions (
    locationid text,
    transactionheaderid text NOT NULL,
    ordersessionid text,
    orderid text,
    orderitemid text,
    menuitemid text,
    modifiergroupid text NOT NULL,
    modifierid text NOT NULL,
    modifiername text,
    parent_modifier_id text,
    nesting_depth integer,
    modifierquantity integer,
    modifierprice numeric(12,3),
    freequantity integer,
    selection_type text,
    action text,
    session_recorded_at text,
```

```
    businessdate date,
    orderdatelocal timestamp without time zone,
    frequentcustomerid text,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    sourceid integer
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_silver_modifier_interactions_to_fact (called by ModifierUpsellRecommendationsToGAS::factModfrInteractions) reads new rows from stg.silver_modifier_interactions past the fact.watermarktable checkpoint, dedups with DISTINCT ON (transactionheaderid, modifiergroupid, modifierid, modifier_interactions_action, modifier_interactions_recorded_at), joins fact.transactionitem, fact.itemmodifier, dim.modifier_group_mapping and dim.modifier_group, and inserts via a double NOT EXISTS anti-join (it also references an upsellinformation.modifierinteractions table outside the dim/fact/etl/ml/stg schema set, presumably a source-side schema exposed via a foreign table or cross-database reference).

fact.usp_modifier_interaction_analysis (called by the same pipeline's StgToFact step) is a second contributor, deriving additional interaction rows from fact.modifier_recommendations and fact.itemmodifier.

fact.usp_modifier_recommendation_analysis (not currently wired to any pipeline call) also targets this table as part of its broader impression/interaction/recommendation logic.

Also fed directly by the gasOrderItemsFromCosmosToAnalyticsDb data flow.

5.14 fact.modifier_recommendations

One row per (location, order) modifier-recommendation session summary — the session-level record that the impression and interaction analysis procedures both derive additional rows from.

DDL

```
CREATE TABLE fact.modifier_recommendations (
    locationid text NOT NULL,
    transactionheaderid text NOT NULL,
    ordersessionid text,
    orderid text,
    modifier_impressions jsonb,
    modifier_interactions jsonb,
    businessdate date,
    orderdateutc text,
    orderdatelocal timestamp without time zone,
    frequentcustomerid text,
```

```
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_modifier_recommendations_syscosmosts_brin	brin	syscosmosts

How It's Populated

fact.usp_silver_modifier_recommendations_to_fact (called by ModifierUpsellRecommendationsToGAS::factModfrRecommendations, the first step in that pipeline) reads new rows from stg.silver_modifier_recommendations past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, transactionheaderid), joins fact.transactionheader, and inserts via a NOT EXISTS anti-join.

A BRIN index on syscosmosts supports the incremental range scan.

5.15 fact.occasionsurveydetail

One row per (order, survey) occasion-level guest-feedback response — the order-level counterpart to fact.itemssurvey's item-level detail.

DDL

```
CREATE TABLE fact.occasionsurveydetail (
    organizationid text,
    locationid text,
    dateid integer,
    surveyid text,
    surveytransid text,
    orderid text,
    surveyrating text,
    surveytransstatus text,
    surveyissuedtimestamp text,
    surveycompletedtimestamp text,
    surveylocaltimestamp timestamp without time zone,
    sysinserttime timestamp without time zone,
    syscosmosts bigint,
    sourceid integer,
    surveytype integer,
    ordersessionid text
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
FOREIGN KEY	locationid_transactionheaderid_fk	locationid, orderid

Type	Constraint Name	Column(s)
FOREIGN KEY	orgid_locationid_fk	organizationid, locationid
FOREIGN KEY	sourceid_fk	sourceid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_stg_occasionsurveydetail_to_fact (called by OccasionSurveyOrdersFeedback::usp_stg_occasionsurveydetail_to_fact) reads new rows from stg.fact_occasionsurveydetail past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, orderid, surveyid, surveytransid), joins fact.transactionheader, dim.organizationlocation, dim.occasionsurvey and stg.silver_kiosk_events (for token-based session matching), and inserts via a double NOT EXISTS anti-join.

fact.usp_update_occasion_survey_datetime_fields subsequently backfills business-local date/time fields by joining dim.organizationlocation/dim.location/dim.organization/dim.occasionsurvey.

Also fed directly by the ngFactOccasionSurveyDetail data flow (which also populates fact.sent_surveys in the same run). Carries (disabled/Citus-incompatible) logical foreign keys on (locationid, orderid), (organizationid, locationid) and sourceid.

5.16 fact.ordertiming

One row per order-session timing record (session start/end, per-stage durations across the ordering flow) — a wide (37-column) table used for kiosk ordering-flow performance analysis.

DDL

```
CREATE TABLE fact.ordertiming (
    id bigint DEFAULT nextval('fact.ordertiming_id_seq'::regclass) NOT NULL,
    companyid text,
    locationid text,
    eventtoken text,
    dateid integer,
    deviceid text,
    sessionstart timestamp without time zone,
    menustart timestamp without time zone,
    itemstart timestamp without time zone,
    checkoutstart timestamp without time zone,
    paymentstart timestamp without time zone,
    paymentend timestamp without time zone,
    orderend timestamp without time zone,
    starttomenu NUMERIC(10,3),
    menutoitem NUMERIC(10,3),
    itemtocheckout NUMERIC(10,3),
    checkouttopayment NUMERIC(10,3),
    paytopaid NUMERIC(10,3),
    payendtoend NUMERIC(10,3),
    starttocheckout NUMERIC(10,3),
```

```
    checkouttoend NUMERIC(10,3),
    totalordertime NUMERIC(10,3),
    sysinserttime timestamp without time zone,
    syscosmosts bigint,
    sysupdatetime TIMESTAMP,
    CONSTRAINT ordertiming_pkey PRIMARY KEY (id),
    CONSTRAINT locationid_eventtoken_unq UNIQUE (locationid, eventtoken)
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	ordertiming_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_gem_ordertiming_to_fact_ordertiming (called by gemOrderTimingtoAnalyticsDb::gemOrderTimingToFact) reads new rows from stg.silver_kiosk_events past the fact.watermarktable checkpoint and performs an INSERT ... ON CONFLICT (locationid, eventtoken, deviceid, sessionstart) DO UPDATE, i.e. it maintains one evolving row per session as later timing events arrive for the same session.

Also fed directly by the gemOrderTimingFromCosmosToPG data flow (a parallel Cosmos-direct path). A unique constraint on (locationid, eventtoken) alongside the primary key on id documents the natural key used by the ON CONFLICT logic.

5.17 fact.peripheralhealth

One row per peripheral-device health-check record (printer, scanner, etc.) — the peripheral-level companion to fact.devicehealth.

DDL

```
CREATE TABLE fact.peripheralhealth (
    healthdataid bigint,
    peripheralid character varying(50) NOT NULL,
    peripheraltype character varying(50) NOT NULL,
    status character varying(50) NOT NULL,
    statusmessage text
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	peripheralhealth_healthdataid_peripheralid_key	healthdataid, peripheralid
FOREIGN KEY	peripheralhealth_healthdataid_fkey	healthdataid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

Populated directly by the gshDevicesAndPeripheralsToAnalyticsDb and gshPeripheralsToAnalyticsDb data flows, both sourced from gsh.peripheralhealth. No PL/pgSQL stored procedure targets this table in the reviewed source.

Carries a unique constraint on (healthdataid, peripheralid) and a (disabled/Citus-incompatible) logical foreign key on healthdataid.

5.18 fact.peripheralstate

By naming convention, a peripheral-level state snapshot table paralleling fact.devicestate, though no ADF or PL/pgSQL evidence of its use was found.

DDL

```
CREATE TABLE fact.peripheralstate (
    deviceid text,
    peripheralid text,
    peripheraltype text,
    state text,
    statestart timestamp with time zone,
    stateend timestamp with time zone,
    duration interval
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts references this table (unlike fact.peripheralhealth, which is actively populated). It may be a planned-but-unbuilt table, or populated by a process outside the reviewed ADF factory / SQL source — recommend confirming with the team.

5.19 fact.pipelinerunstatus

One row per ADF pipeline run (pipeline name, run id, trigger time, completion time, success flag, error message) — the operational run-log for the whole factory, keyed loosely by pipelinename and correlationid.

DDL

```
CREATE TABLE fact.pipelinerunstatus (
    pipelinename text NOT NULL,
    pipelinerunid character varying(100) NOT NULL,
    pipelinetriggertime timestamp without time zone NOT NULL,
    issuccess boolean,
    pipelinecompletedtime timestamp without time zone,
    correlationid character varying(100),
    pipelinestatus character varying(50),
    pipelinemessage text,
    triggeredbyuserid character varying(100)
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_fact_pipelinerunstatus_correlationid	btree	correlationid

How It's Populated

The initial 'Running' row for a pipeline execution is not created by any Lookup/Script/Copy activity captured in this ARM export — it is most likely inserted by an external event-driven mechanism (an Azure Function or Logic App subscribed to ADF pipeline-run start events), consistent with the ADFEventLoggingTemplate pipeline's use of WebActivity calls (RunStarted/RunSucceeded/RunFailed) rather than direct SQL.

What every major pipeline does contain, however, is a pair of 'SuccessStatus'/'FailStatus' Lookup activities near the end of the run (44 such activities found across the 67 pipelines) that UPDATE this table's issuccess, pipelinestatus, pipelinecompletedtime and pipelinerunid columns, matched either on pipelinename (when correlationid is null) or on pipelinerunid (when correlationid is present) — i.e. this table is written from inside almost every pipeline, but only ever via UPDATE, never INSERT, from what's visible in the ARM template.

An index on correlationid supports the run-linking lookup used by nested/child pipeline executions.

5.20 fact.pos_sales_details

By naming convention, a POS-system sales-detail fact table (13 columns, its own primary key on id).

DDL

```
CREATE TABLE fact.pos_sales_details (
    id bigint NOT NULL,
    businessdate date NOT NULL,
    posorderplaced bigint NOT NULL,
    possales numeric(7,3) DEFAULT 0 NOT NULL,
    postips numeric(7,3) DEFAULT 0,
    locationid text NOT NULL,
    created_on timestamp without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
    created_by character varying(255),
    modified_on timestamp without time zone DEFAULT CURRENT_TIMESTAMP,
    modified_by character varying(255),
    is_active boolean DEFAULT false NOT NULL
```

```
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	pos_sales_details_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts references this table. It appears to be either a legacy table from an earlier direct-POS integration that has since been replaced by the Cosmos/GMS-based transaction pipeline (fact.transactionheader/transactionitem/transactionpayment), or a table reserved for a POS integration not yet built out in this factory export.

5.21 fact.recommendations

One row per (location, order, recommendation) upsell-recommendation event — records what was recommended and whether/what was selected; the base table behind fact.vw_offer_analysis's funnel breakdown.

DDL

```
CREATE TABLE fact.recommendations (
    transactionheaderid character varying(50) NOT NULL,
    locationid character varying(50) NOT NULL,
    recommendationid character varying(50) NOT NULL,
    offereditems jsonb,
    selecteditems jsonb,
    isconverted boolean,
    prompttimestamp text,
    sysinserttime timestamp without time zone,
    syscosmosts bigint
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	locationid_trxnid_recommendationid_pk	locationid, transactionheaderid, recommendationid
UNIQUE	transactionheaderid_recommendationid_uidx	transactionheaderid, recommendationid
FOREIGN KEY	location_transactionheaderid_fk	locationid, transactionheaderid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
ix_upsell_recommendations_syscosmosts_brin	brin	syscosmosts

How It's Populated

Two independent procedures feed this table. fact.usp_item_recommendations_stage_to_fact (called by UpsellRecommendationsToGAS::StgToFact) reads stg.recommendations and inserts new rows via a NOT EXISTS anti-join.

fact.usp_silver_upsell_recommendations_to_fact (called by the same pipeline's factUpsellRecommendations step) separately reads new rows from stg.silver_upsell_recommendations past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, transactionheaderid, recommendationid), joins fact.transactionheader, and performs an INSERT ... ON CONFLICT (locationid, transactionheaderid, recommendationid) DO UPDATE — i.e. the Bronze/silver path and the Cosmos-direct stg.recommendations path both contribute rows to the same fact table via different keys/logic.

A BRIN index on syscosmosts supports incremental scans. Carries a primary key on (locationid, transactionheaderid, recommendationid), a unique constraint on (transactionheaderid, recommendationid), and a (disabled/Citus-incompatible) logical foreign key on (locationid, transactionheaderid).

5.22 fact.recommendations_bkp

Structural backup copy of (an earlier, narrower 7-column version of) fact.recommendations.

DDL

```
CREATE TABLE fact.recommendations_bkp (
    transactionheaderid character varying(50) NOT NULL,
    recommendationid character varying(50) NOT NULL,
    offereditems jsonb,
    selecteditems jsonb,
    isconverted boolean,
    prompttimestamp text,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure targets this table — a manual point-in-time snapshot, consistent with the other '_bkp' tables found in dim.

5.23 fact.sent_surveys

One row per (location, order session) recording that an occasion survey was sent to the guest — the 'was it sent' counterpart to fact.itemssurvey/fact.occasionsurveydetail's 'was it answered' detail.

DDL

```
CREATE TABLE fact.sent_surveys (
    organizationid text,
    locationid text NOT NULL,
    ordersessionid text NOT NULL,
    orderid text,
    gem_event_category text,
    gem_event_type text,
    survey_metadata jsonb,
    is_responded boolean,
    gem_event_instant text,
    gem_syscosmosts bigint,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	sent_surveys_ordersessionid_pkey	locationid, ordersessionid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_gem_sent_surveys_to_fact (called by OccasionSurveyOrdersFeedback::GemSentSurveysToFact) reads new rows from stg.silver_kiosk_events past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, token), joins dim.organizationlocation, and inserts via a NOT EXISTS anti-join.

Also fed directly (for a different survey-delivery channel) by the ngFactOccasionSurveyDetail data flow, which writes to fact.sent_surveys in the same run that it populates fact.occasionsurveydetail from stg.sent_surveys / ngOrdersFeedback.

Primary key on (locationid, ordersessionid) documents the intended one-row-per-session grain.

5.24 fact.timingsdatalake

A minimal two-column table (containername as primary key, plus presumably a last-processed timestamp) that, by naming and shape, looks like an early precursor to etl.bronze_partition_registry — tracking ADLS container-level processing checkpoints rather than hourly partitions.

DDL

```
CREATE TABLE fact.timingsdatalake (
```

```
    containername text NOT NULL,
    timing_value timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	timingsdatalake_pkey	containername

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts references this table. Given etl.bronze_partition_registry is the actively used mechanism for tracking Bronze partition processing today, this table is most likely a retired predecessor left in place.

5.25 fact.transactionheader

One row per order/transaction (60 columns: totals, order type, timing, location, source, service-charge/charity amounts) — the central fact table of the entire warehouse, joined by nearly every other transaction-scoped fact and several dim procedures.

DDL

```
CREATE TABLE fact.transactionheader (
    id bigint DEFAULT nextval('fact.transactionheader_id_seq'::regclass) NOT NULL,
    transactionheaderid text NOT NULL,
    orderid text,
    locationid text NOT NULL,
    kioskid text,
    ordersessionid text,
    dateid integer,
    orderdateutc text,
    orderdatelocal timestamp without time zone,
    orderstatus text,
    ordertype integer,
    numberofitems smallint,
    numberofpayments smallint,
    ordersredeemedrewards numeric(12,3),
    ordersubtotal numeric(12,3),
    ordertotal numeric(12,3),
    ordertax numeric(12,3),
    ordertip numeric(12,3),
    orderdiscount numeric(12,3),
    orderbalance numeric(12,3),
    paymentstatus text,
    sourcefile text DEFAULT 'NGE'::text NOT NULL,
    createddate timestamp without time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
    updateddate timestamp without time zone,
```

```
    orderstarttime timestamp without time zone,
    reviewordertime timestamp without time zone,
    checkouttime timestamp without time zone,
    paystarttime timestamp without time zone,
    sessionendtime timestamp without time zone,
    precheckouttime NUMERIC(10,3),
    postcheckouttime NUMERIC(10,3),
    menupagetime NUMERIC(10,3),
    reviewpagetime NUMERIC(10,3),
    paymentpagetime NUMERIC(10,3),
    totalordertime NUMERIC(10,3),
    businessdate date,
    frequentcustomerid text,
    abtestid bigint,
    channel text,
    guestcount integer,
    charityamount numeric(12,3),
    syscosmosts bigint,
    sourceid integer,
    orderservicecharge numeric(12,3) DEFAULT 0.000,
    customername character varying(100)
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	transactionheader_pkey	locationid, transactionheaderid
FOREIGN KEY	locationid_fk	locationid
FOREIGN KEY	ordertype_fk	ordertype
FOREIGN KEY	sourceid_fk	sourceid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
ix_transactionheader_syscosmosts_brin	brin	syscosmosts
transactionheader_locationid_dateid_idx	btree	locationid, dateid

How It's Populated

fact.usp_silver_transaction_header_to_fact (called by gasTransactionsToAnalyticsDb::factTrxnHeader, the first step of the daily orders chain) reads new rows from stg.silver_transaction_header past the fact.watermarktable checkpoint, also joining stg.silver_kiosk_events for session-token correlation, dedups with DISTINCT ON (locationid, transactionheaderid), joins dim.ordertype and dim.organization, and performs an INSERT ... ON CONFLICT (locationid, transactionheaderid) DO UPDATE.

fact.usp_silver_aborted_orders_and_items_to_fact (called by AbortedOrdersAndItemsToPG, which runs after ModifierUpsellRecommendationsToGAS in the daily Facts chain) is a second, complementary contributor: it mines stg.silver_kiosk_events for kiosk sessions that never produced a completed order document, dedups with a ROW_NUMBER()-based DISTINCT ON (locationid, token), joins dim.kiosk and dim.organization, and INSERTs with ON

CONFLICT (locationid, transactionheaderid) DO NOTHING — recovering abandoned/aborted orders that the main header procedure would never see.

fact.usp_silver_transaction_refunds_to_fact also updates refund-related columns on this table as a side effect while processing stg.silver_transaction_refunds.

fact.usp_update_datetime_fields (called from three different pipelines: AbortedOrdersAndItemsToPG, the standalone UpdateDateTimeFields pipeline, and gasTransactionsToAnalyticsDb — all three under the label 'update businessdate') backfills business-local date/time columns using dim.organization and dim.abtests.

Historically also fed directly by a large family of Cosmos-direct data flows (AbortedEventsFromCosmosGEMToFact, GASOrderHeadersFromCosmosToAnalyticsDb, GXSOrderHeadersFromCosmosToAnalyticsDb, UpdateAmountFieldsInTrxnHeaders, UpdateCharityAmountInTrxns, UpdateFieldsInTrxnFactTables, UpdateHighAmountInTrxnTables, UpdateOrderTimingFieldsInTrxnHeader/UpdateNegativeOrderTimingFieldsInTrxnHeader, UpdateServiceChargeGASOrderHeadersFromCosmosToAnalyticsDb, gasOrderHeadersToAnalyticsDb, and Old/Test variants of several of these) — a large surface of parallel/legacy correction and enrichment paths that predate or supplement the current Bronze/silver-based stored-procedure pipeline.

A BRIN index on syscosmosts and a composite (locationid, dateid) index support the very high incremental-scan and reporting-join volume against this table. Carries a primary key on (locationid, transactionheaderid) and (disabled/Citus-incompatible) logical foreign keys on locationid, ordertype and sourceid.

5.26 fact.transactionitem

One row per order line item (31 columns: item id/name, quantity, price, combo linkage, category) — the second-most-joined fact table after fact.transactionheader.

DDL

```
CREATE TABLE fact.transactionitem (
    transactionheaderid text NOT NULL,
    categoryid bigint,
    menuitemid bigint,
    itemid text NOT NULL,
    comboid text,
    ordersessionid text NOT NULL,
    itemsessionid text,
    itemname text NOT NULL,
    itemquantity smallint DEFAULT 1,
    itemunitprice numeric(12,3),
    upselllevel text,
    upsellpromptitemid text,
    orderid text NOT NULL,
    itemtype text,
    customize boolean,
    upgrade boolean,
    asis boolean,
    itemselectedtime timestamp without time zone,
    addtocarttime timestamp without time zone,
    totaltime NUMERIC(10,3),
    orderdateutc text,
    sysinserttime timestamp without time zone,
    sysupdatetime timestamp without time zone,
    dimmenuitemid character varying(50),
    locationid character varying(50),
```

```
    orderdatelocal timestamp without time zone,
    businessdate date,
    syscosmosts bigint,
    frequentcustomerid text
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	transactionitem_pkey	transactionheaderid, itemid, itemname
FOREIGN KEY	categoryid_fk	categoryid
FOREIGN KEY	locationid_fk	locationid
FOREIGN KEY	locationid_transactionheaderid_fk	locationid, transactionheaderid
FOREIGN KEY	menuitemid_fk	menuitemid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
idx_fact_transactionitem_locationid	btree	locationid

How It's Populated

fact.usp_silver_transaction_item_to_fact (called by gasTransactionsToAnalyticsDb::factTrxnItem, right after factTrxnHeader) reads new rows from stg.silver_transaction_item and stg.silver_transaction_combo_items past the fact.watermarktable checkpoint (also consulting stg.silver_kiosk_events for session correlation), dedups line items and combo/component rows with several separate DISTINCT ON clauses, joins dim.menuitem, dim.itemcategory, fact.transactionheader and dim.organization, and performs an INSERT ... ON CONFLICT (transactionheaderid, itemid, itemname) DO UPDATE.

fact.usp_silver_aborted_orders_and_items_to_fact contributes recovered line items for aborted/abandoned kiosk sessions in the same run that it recovers fact.transactionheader rows, with its own ON CONFLICT (transactionheaderid, itemid, itemname) DO NOTHING guard.

fact.usp_update_datetime_fields also backfills date/time columns here alongside fact.transactionheader.

Historically also fed directly by the same large family of Cosmos-direct data flows noted under fact.transactionheader (AbortedItemsFromCosmosGEMToFact, GXSgasOrderItemsFromCosmosToAnalyticsDb, UpdateFieldsInTrxnFactTables, UpdateItemsAndPaymentsWithHighAmounts, gasOrderItemsFromCosmosToAnalyticsDb, plus the GEM-user-behaviour ETL family), and referenced read-only by dim.usp_master_keys_for_duplicate_items and several ml.usp_refresh_* procedures.

An index on locationid supports location-scoped reporting. Carries a primary key on (transactionheaderid, itemid, itemname) and (disabled/Citus-incompatible) logical foreign keys on categoryid, locationid, (locationid, transactionheaderid) and menuitemid.

5.27 fact.transactionpayment

One row per payment applied to an order (tender type, amount, payment transaction id) — child of fact.transactionheader.

DDL

```
CREATE TABLE fact.transactionpayment (
    transactionheaderid text NOT NULL,
    paymentintegrationid text NOT NULL,
    paymentid text,
    paymentamt numeric(12,3),
    orderid text NOT NULL,
    locationid character varying(50),
    kioskid character varying(50),
    paymentmethod character varying(50),
    paymentintegrationlabel text,
    orderdateutc text,
    sysinserttime timestamp without time zone,
    paymentcardtype character varying(50),
    sysupdatetime timestamp without time zone,
    syscosmosts bigint
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
FOREIGN KEY	`locationid_transactionheaderid_fk`	`locationid, transactionheaderid`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
`idx_fact_transactionpayment_locationid`	btree	`locationid`

How It's Populated

fact.usp_silver_transaction_payment_to_fact (called by gasTransactionsToAnalyticsDb::factTrxnPayment) reads new rows from stg.silver_transaction_payment past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, transactionheaderid, payment_transactionid), joins fact.transactionheader, and inserts via a NOT EXISTS anti-join.

Also updated as a side effect by fact.usp_silver_transaction_refunds_to_fact when a refund is linked to a payment, and historically fed directly by the GXSgasOrderItemsFromCosmosToAnalyticsDb, UpdateItemsAndPaymentsWithHighAmounts, UpdateTrxnPayments and gasOrderItemsFromCosmosToAnalyticsDb data flows.

An index on locationid supports location-scoped reporting. Carries a (disabled/Citus-incompatible) logical foreign key on (locationid, transactionheaderid).

5.28 fact.transactionrefunds

One row per refund issued against an order (amount, reason, linked payment) — child of fact.transactionheader.

DDL

```
CREATE TABLE fact.transactionrefunds (
    transactionheaderid character varying(50) NOT NULL,
    orderid character varying(50),
    locationid character varying(50),
    refundtransactionid character varying(50),
    paymentid character varying(50),
    refundamount numeric(7,3),
    refundtype character varying(50),
    orderdateutc text,
    sysinserttime timestamp without time zone,
    syscosmosts bigint
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
idx_transactionrefunds_headerid	btree	transactionheaderid

How It's Populated

fact.usp_silver_transaction_refunds_to_fact (called by gasTransactionsToAnalyticsDb::factTrxnRefunds, the last step of the daily orders chain before the modifier/upsell pipelines) reads new rows from stg.silver_transaction_refunds past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, transactionheaderid), joins fact.transactionheader and fact.transactionpayment, and inserts via a NOT EXISTS anti-join (also updating linked fact.transactionheader/fact.transactionpayment amount columns as a side effect).

Also fed directly by the gasTransactionRefunds data flow. An index on transactionheaderid supports the join back to the header table.

5.29 fact.userbehaviour

One row per guest UI interaction/clickstream event during a kiosk session (screen, element, order type, view context) — the richest behavioral-analytics fact table, feeding the ML upsell/preference models.

DDL

```
CREATE TABLE fact.userbehaviour (
    id bigint DEFAULT nextval('fact.userbehaviour_id_seq'::regclass) NOT NULL,
    busdate timestamp without time zone,
    locationid text,
    dateid integer,
    daypart text,
    ordertype bigint,
    eventtype text,
    ordersessionidentifier text,
    viewidentifier integer,
    itemsessionidentifier text,
    elementidentifier integer,
    quantity integer,
```

```
    createddate timestamp without time zone,
    syscosmosts bigint,
    eventinstant text,
    eventcategory text,
    sysupdatetime timestamp without time zone,
    deviceid TEXT,
    syscosmosticks BIGINT,
    eventdata TEXT
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	userbehaviour_pkey	id

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

Index Name	Method	Column(s)
ix_userbehaviour_createddate_brin	brin	createddate
ix_userbehaviour_syscosmosts_brin	brin	syscosmosts
userbehaviour_locationid_dateid_idx	btree	locationid, dateid

How It's Populated

fact.usp_silver_kiosk_events_to_fact_userbehaviour (called by gemEventsToAnalyticsDb::factUserBehaviour, the last step of that pipeline) reads new rows from stg.silver_kiosk_events past the fact.watermarktable checkpoint, also cross-referencing stg.silver_transaction_header, dedups with a COALESCE(NULLIF(token,...)) DISTINCT ON expression, joins dim.ordertype, dim.view and dim.element (discovering/creating new lookup values in the latter two via their own refresh procedures), and inserts using a triple NOT EXISTS anti-join.

Historically also fed directly by a family of Cosmos-direct data flows (EventsFromCosmosGEMToFact / MergedEventsFromCosmosGEMToFact / ReloadGemUserBehaviourEventsToFact / TestGEMUserBehaviourEventsToFact / gemUserBehaviourEventsToFact), all of which join gemEvent documents against dim.location, fact.transactionheader, dim.view, dim.element, dim.ordertype and fact.transactionitem in various combinations.

Three indexes — BRIN on createddate, BRIN on syscosmosts, and a composite (locationid, dateid) — reflect this table's very high row volume and its dual access pattern (time-range scans plus location/day rollups).

5.30 fact.userbehaviour_exceptions

By naming and column count (13, mirroring much of fact.userbehaviour's shape), an error/exception-capture table for rows that failed validation during the userbehaviour ETL.

DDL

```
CREATE TABLE fact.userbehaviour_exceptions (
```

```
    id bigint NOT NULL,
    busdate timestamp without time zone,
    locationid text,
    dateid integer,
    daypart text,
    ordertype bigint,
    eventtype text,
    ordersessionidentifier text,
    viewidentifier integer,
    itemsessionidentifier text,
    elementidentifier integer,
    quantity integer,
    createddate timestamp without time zone
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	`userbehaviour_exceptions_pkey`	`id`

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

No Copy Activity, ExecuteDataFlow, or stored procedure in the reviewed artifacts explicitly targets this table. It is most likely written to from inside one of the GEM-user-behaviour data flows' reject/error branch (data flows can route failed rows to a separate sink that isn't captured by this extraction's source/sink resolution), rather than from a stored procedure.

5.31 fact.usercheckedin

One row per (location, order) guest check-in event (used for dine-in/curbside/table-service flows where the guest checks in before or after ordering).

DDL

```
CREATE TABLE fact.usercheckedin (
    organizationid text NOT NULL,
    locationid text NOT NULL,
    kioskid text,
    ordersessionid text,
    dateid integer,
    ordertimestamp text,
    orderid text NOT NULL,
    customername text,
    customerphone text,
    orderstatus text,
    ordertotal numeric(7,3),
    paymentstatus text,
    amountpaid numeric(7,3),
```

```
    paymentmethod text,
    paymentcardtype text,
    sysinserttime timestamp without time zone,
    orderdatelocal timestamp without time zone,
    syscosmosts bigint
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	usercheckedin_pkey	locationid, orderid

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_gem_usercheckedin_to_fact_usercheckedin (called by gemUserCheckedInToPG::stgUserCheckedinToFact, which runs right after gemOrderTiming in the GEM_GSH_GOU_Master chain) reads new rows from stg.silver_kiosk_events past the fact.watermarktable checkpoint, dedups with DISTINCT ON (locationid, orderid), joins dim.location and dim.organization, and performs an INSERT ... ON CONFLICT (locationid, orderid) DO UPDATE.

Also fed directly by the gemUserCheckedInToPG data flow (the primary/current path, joining gemEvent against dim.location).

5.32 fact.vw_offer_analysis

One row per (order, recommendation, offered item) upsell-offer outcome (offered item vs. selected item) — despite the 'vw_' naming this is a physical TABLE, the funnel-analysis output built from fact.recommendations.

DDL

```
CREATE TABLE fact.vw_offer_analysis (
    locationid character varying(50) NOT NULL,
    transactionheaderid character varying(50) NOT NULL,
    recommendationid character varying(50) NOT NULL,
    offereditem character varying(50) NOT NULL,
    selecteditem character varying(50),
    upselltype character varying(50),
    upsellgroupid character varying(50),
    upsellgroupname text,
    quantity integer,
    prompttimestamp text,
    upsellprompttime timestamp without time zone,
    syscosmosts bigint,
    sysinserttime timestamp without time zone,
    offereditem_upselllevel TEXT,
    offered_promptitemid TEXT,
    offered_upsellgroupid TEXT,
    selecteditem_upselllevel TEXT,
    selected_promptitemid TEXT,
```

```
    selected_upsellgroupid TEXT
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
UNIQUE	trxnid_recommendationid_itemid_uidx	transactionheaderid, recommendationid, offereditem
FOREIGN KEY	locationid_trxnid_recommendationid_fk	locationid, transactionheaderid, recommendationid
FOREIGN KEY	selecteditem_fk	selecteditem

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

fact.usp_offer_analysis (called by UpsellRecommendationsToGAS::Run_uspOfferAnalysis, the last step of that pipeline) reads new rows from fact.recommendations past the fact.watermarktable checkpoint, joins dim.upsellgrouplookup and dim.category_hierarchy for enrichment, and inserts via a NOT EXISTS anti-join.

The UpdateUpsellRecommendations data flow separately reads this table joined to public.upsell_group to refresh dim.upsellgrouplookup's derived attributes — a read (not write) dependency in the other direction.

Carries a unique constraint on (transactionheaderid, recommendationid, offereditem) and (disabled/Citus-incompatible) logical foreign keys on (locationid, transactionheaderid, recommendationid) and selecteditem.

5.33 fact.watermarktable

The central incremental-load checkpoint table for the whole warehouse: one row per (watermarktablename, source), storing the highest syscosmosts (or equivalent) value already processed into that target. This is what makes almost every fact.usp_* procedure incremental rather than full-refresh.

DDL

```
CREATE TABLE fact.watermarktable (
    watermarktablename text NOT NULL,
    watermarkcolumn text,
    watermarkvalue timestamp without time zone,
    ticks bigint,
    ts bigint,
    source character varying(50) NOT NULL,
    sysinserttime TIMESTAMP,
    sysupdatetime TIMESTAMP
);
```

Constraints & Relationships

Type	Constraint Name	Column(s)
PRIMARY KEY	watermarktablename_pk	watermarktablename, source

Citus (the distributed PostgreSQL engine gas_db runs on) does not enforce most primary-key / unique / foreign-key constraints across distributed tables. These are documented as the logical key design captured in the source DDL, not necessarily as active database-level constraints.

Indexes

No dedicated index was found for this table beyond its default storage order.

How It's Populated

This is the single most heavily used control table in gas_db: essentially every fact.usp_silver_*_to_fact, fact.usp_gem_*, fact.usp_gsh_*, fact.usp_modifier_*_analysis and related procedure reads its own row here at the start of a run (WHERE watermarktablename = '<target>' AND source = '<source>') to know where to resume, and either calls fact.updatewatermark(tablename) or issues its own UPDATE at the end of the run to advance the checkpoint to MAX(syscosmosts) of the rows just processed.

fact.updatewatermark(tablename) is a small helper function that centralizes that end-of-run update for callers that use it directly (its own analysis shows it reading fact.devicestate's max syscosmosts as an example of the pattern); most of the fact.usp_* procedures now advance the watermark inline via their own UPDATE statement rather than calling this helper.

Because every downstream fact procedure both reads and writes this table, it is effectively the dependency-ordering pivot of the entire fact-layer run: a stalled or incorrect watermark for one target only affects that target's incremental window, not any other table's, since each row is scoped independently by (watermarktablename, source).

Its primary key (watermarktablename, source) is one of the very few constraints in the whole warehouse that would be safe to enforce as a true unique index even under Citus, since lookups are always by that exact composite key.

6. ML Schema - Machine Learning Feature Tables

Eight tables rebuilt daily from the dim/fact Gold layer as feature tables for the Data Science team's models.

6.1 ml.item_modifiergroup_modifier_mapping

One row per (location, menu item, modifier group, modifier) — the ML-ready, fully denormalized feature table describing which modifiers are available on which item, joined with catalog/organization context. Feeds both the upsell/interaction models and ml.usp_refresh_item_modifier_matching's fuzzy-matching pass.

DDL

```
CREATE TABLE ml.item_modifiergroup_modifier_mapping (
    organizationid text,
    organizationname text,
    locationid text,
    locationname text,
    catalogid text,
    catalogname text,
    menuitemid text,
    menuitemname text,
    item_class_type integer,
```

```
    modifiergroupid text,
    modifiergroupname text,
    modifierid text,
    modifiername text,
    modifier_class_type integer,
    is_modifier_default boolean,
    min_quantity integer,
    max_quantity integer,
    allow_quantity_increment boolean,
    increment_step integer,
    modifier_default_quantity integer,
    is_modifier_invisible boolean,
    calories text,
    price numeric(12,4),
    is_modifier_active boolean,
    is_modifier_deleted boolean,
    modifier_created_on timestamp without time zone,
    modifier_modified_on timestamp without time zone,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_imm_locationid	btree	locationid

How It's Populated

ml.usp_refresh_item_modifiergroup_modifier_mapping(p_organizationid) is called by RefreshMLTables::ModifierEntitiesML (second step of the daily ML refresh chain, right after MenuEntitiesML) and also by ChildPipTrainingDataML::GetLocationAndItemStatistics for training-data snapshot preparation.

It DELETEs existing rows for the given organization from this table and rebuilds them from dim.organizationlocation, dim.modifier and dim.item_modifier_group_modifier_mapping, joined to dim.catalog, dim.menuitem and dim.modifier_group — a full per-organization refresh rather than a watermark-incremental load.

An index on locationid supports the location-scoped feature extraction queries run against this table by downstream ML training jobs.

6.2 ml.menu_entities

One row per (location, menu item) with category-hierarchy context flattened in — the base menu-item feature table joined by nearly every other ml.* refresh procedure and by ml.usp_refresh_item_modifier_matching.

DDL

```
CREATE TABLE ml.menu_entities (
    organizationid text,
    organizationname text,
    locationid text,
    locationname text,
```

```
    categoryid text,
    categoryname text,
    menuitemid text,
    menuitemname text,
    catalogid text,
    itemunitprice numeric(12,3),
    price_changed_on timestamp without time zone,
    item_class_type integer,
    entitytype text,
    calories text,
    protein numeric(9,2),
    sugar numeric(9,2),
    fat numeric(9,2),
    is_alcoholic boolean,
    is_vegetarian_item boolean,
    is_vegan_item boolean,
    has_allergen boolean,
    is_active boolean,
    is_deleted boolean,
    gms_created_on timestamp without time zone,
    gms_modified_on timestamp without time zone,
    sysinserttime timestamp without time zone,
    average_rating NUMERIC(3,2),
    rating_count INTEGER
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_me_locationid	btree	locationid
ix_ml_me_menuitemid	btree	menuitemid

How It's Populated

ml.usp_refresh_menu_entities is called by RefreshMLTables::MenuEntitiesML, the first table-building step of the daily ML refresh chain (right after WeatherML).

It TRUNCATEs this table and rebuilds it in full from dim.menuitem and dim.organizationlocation joined to dim.category_hierarchy.

Two indexes — locationid and menuitemid — support the very frequent joins from ml.usp_refresh_modifier_impressions, ml.usp_refresh_modifier_interactions, ml.usp_refresh_transactions and ml.usp_refresh_item_modifier_matching.

6.3 ml.modifier_impressions

One row per (location, week, item, modifier) modifier-impression feature aggregate (35 columns) — the ML-ready weekly rollup built from fact.modifier_impressions.

DDL

```
CREATE TABLE ml.modifier_impressions (
    organizationid text,
    organizationname text,
    locationname text,
    locationid text,
    catalogid text,
    catalogname text,
    businessdate date,
    orderdatelocal timestamp without time zone,
    yyyy integer,
    ww integer,
    transactionheaderid text,
    ordersessionid text,
    orderid text,
    menuitemid text,
    menuitemname text,
    item_class_type integer,
    modifierid text,
    modifiername text,
    modifier_class_type integer,
    parent_modifier_id text,
    nesting_depth integer,
    modifierprice numeric(12,3),
    selection_type text,
    "position" integer,
    score numeric(10,4),
    strategy text,
    context text,
    selected boolean,
    pre_deselected boolean,
    confirmed_removed boolean,
    pre_selected boolean,
    frequentcustomerid text,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_mimp_locid_yyyy_ww	btree	locationid, yyyy, ww
ix_ml_modfr_impressions_businessdate	btree	businessdate

How It's Populated

ml.usp_refresh_modifier_impressions(p_refresh_mode) is called by RefreshMLTables::ModifierImpressionsML, near the end of the daily ML refresh chain (after ModifierInteractionsML).

Depending on p_refresh_mode, it DELETEs/TRUNCATEs and rebuilds this table from fact.modifier_impressions joined to dim.organizationlocation, dim.modifier, dim.catalog and dim.menuitem, aggregating to the (location, year, week) grain.

Two indexes — composite (locationid, yyyy, ww) and businessdate — support the time-windowed feature-extraction queries used by model training.

6.4 ml.modifier_interactions

One row per (location, week, item, modifier) modifier-interaction feature aggregate (40 columns, the widest of the weekly rollups) — built from fact.itemmodifier and ml.transactions.

DDL

```
CREATE TABLE ml.modifier_interactions (
    organizationid text,
    organizationname text,
    locationname text,
    locationid text,
    catalogid text,
    catalogname text,
    businessdate date,
    orderdatelocal timestamp without time zone,
    yyyy integer,
    ww integer,
    transactionheaderid text,
    ordersessionid text,
    orderid text,
    orderitemid text,
    menuitemid text,
    menuitemname text,
    itemquantity integer,
    itemunitprice numeric(12,3),
    item_class_type integer,
    modifiergroupid text,
    modifiergroupname text,
    modifierid text,
    modifiername text,
    parent_modifier_id text,
    nesting_depth integer,
    modifierquantity integer,
    modifierprice numeric(12,3),
    freequantity integer,
    is_modifier_default boolean,
    min_quantity integer,
    max_quantity integer,
    selection_type text,
    action text,
    session_recorded_at text,
    frequentcustomerid text,
    modifier_default_quantity integer,
    modifier_class_type integer,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_mi_locid_yyyy_ww	btree	locationid, yyyy, ww
ix_ml_modfr_interactions_businessdate	btree	businessdate

How It's Populated

ml.usp_refresh_modifier_interactions(p_refresh_mode) is called by RefreshMLTables::ModifierInteractionsML, run after UpsellAnalysisML in the daily ML chain.

Depending on p_refresh_mode, it DELETEs/TRUNCATEs and rebuilds this table from ml.transactions and fact.itemmodifier joined to dim.organizationlocation, dim.modifier, dim.catalog, dim.menuitem, dim.modifier_group_mapping and dim.modifier_group, aggregating to the (location, year, week) grain.

Two indexes — composite (locationid, yyyy, ww) and businessdate — mirror the pattern used across the ml.* weekly rollup tables.

6.5 ml.modifier_item_match

One row per (organization, location, catalog, modifier) fuzzy-matched menu item — the output of the token-sort-ratio-based similarity matching used to reconcile modifier names against canonical menu-item names across catalogs (the trigram/TSR fuzzy-dedup logic Nodir ported from Python's rapidfuzz).

DDL

```
CREATE TABLE ml.modifier_item_match (
    organizationid text COLLATE pg_catalog."default" NOT NULL,
    locationid text COLLATE pg_catalog."default" NOT NULL,
    catalogid text COLLATE pg_catalog."default" NOT NULL,
    modifierid text COLLATE pg_catalog."default" NOT NULL,
    modifiername text COLLATE pg_catalog."default" NOT NULL,
    matched_menuitemid text COLLATE pg_catalog."default" NOT NULL,
    matched_menuitemname text COLLATE pg_catalog."default" NOT NULL,
    tsr_score numeric(5,2) NOT NULL,
    match_confidence_tier text COLLATE pg_catalog."default" NOT NULL,
    match_direction text COLLATE pg_catalog."default",
    matched_menuitems jsonb NOT NULL DEFAULT '[]'::jsonb,
    modifiergroup_occurrence_count integer,
    modifier_price numeric(12,4),
    item_price numeric(12,3),
    price_delta numeric(14,4),
    item_entity_type text COLLATE pg_catalog."default",
    item_calories text COLLATE pg_catalog."default",
    item_categoryid text COLLATE pg_catalog."default",
    item_categoryname text COLLATE pg_catalog."default",
    item_is_alcoholic boolean,
    item_is_vegetarian boolean,
    item_is_vegan boolean,
    item_has_allergen boolean,
    item_average_rating numeric(3,2),
    item_rating_count integer,
    modifier_is_default boolean,
    modifier_calories text COLLATE pg_catalog."default",
    modifier_max_quantity integer,
    modifier_min_quantity integer,
    is_size_variant boolean NOT NULL DEFAULT false,
    matched_at timestamp with time zone DEFAULT now(),
    pipeline_version text COLLATE pg_catalog."default" DEFAULT 'v1'::text,
    CONSTRAINT modifier_item_match_pkey PRIMARY KEY (organizationid, locationid, catalogid,
modifierid)
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_mim_catalogid	btree	catalogid COLLATE pg_catalog."default" ASC NULLS LAST
ix_ml_mim_locationid	btree	locationid COLLATE pg_catalog."default" ASC NULLS LAST
ix_ml_mim_tsr_score	btree	organizationid COLLATE pg_catalog."default" ASC NULLS LAST, tsr_score DESC NULLS FIRST

How It's Populated

ml.usp_refresh_item_modifier_matching(p_organizationid, p_tsr_threshold DEFAULT 70, p_trgm_prefilter DEFAULT 0.40) is called by ChildPipTrainingDataML::uspItemModifierSimilarityMatch.

It DELETEs existing rows for the organization and rebuilds from ml.item_modifiergroup_modifier_mapping, dim.organizationlocation and ml.menu_entities, using PostgreSQL's pg_trgm similarity() as a cheap prefilter (p_trgm_prefilter) before computing the more expensive token-sort-ratio score, then keeping matches at or above p_tsr_threshold, and performs an INSERT ... ON CONFLICT (organizationid, locationid, catalogid, modifierid) DO UPDATE keyed on menuitemid dedup (DISTINCT ON menuitemid).

Three indexes — catalogid, locationid, and a composite (organizationid, tsr_score DESC) — support both lookup by catalog/location and the 'best match first' ranking query pattern used when reviewing match quality.

6.6 ml.transactions

One row per (location, week, order) transaction feature record (44 columns spanning order, item, weather and order-type context) — the central ML feature table joined by ml.usp_refresh_modifier_interactions, ml.usp_refresh_upsell_analysis and location-statistics reporting.

DDL

```
CREATE TABLE ml.transactions (
    frequentcustomerid text,
    organizationid text,
    organizationname text,
    locationid text,
    locationname text,
    kioskid text,
    transactionheaderid text,
    ordersessionid text,
    orderid text,
    orderitemid text,
    menuitemid text,
    itemname text,
    upselllevel text,
    item_class_type integer,
    itemquantity integer,
    categoryid text,
    categoryname text,
    itemunitprice numeric(12,3),
    paymentstatus text,
```

```
    numberofitems integer,
    numberofpayments integer,
    ordertotal numeric(14,4),
    ordersubtotal numeric(14,4),
    ordertip numeric(14,4),
    ordertax numeric(14,4),
    ordertypelabel text,
    orderdatelocal timestamp without time zone,
    businessdate date,
    weatherhumidity numeric(7,2),
    weathercondition text,
    temperatureincelcius numeric(7,2),
    yyyy integer,
    mm integer,
    dd integer,
    hh integer,
    ww integer,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_trx_locid_yyyy_ww	btree	locationid, yyyy, ww
ix_ml_trxn_businessdate	btree	businessdate
ix_ml_trx_org_loc_item	btree	organizationid, locationid, menuitemid

How It's Populated

ml.usp_refresh_transactions(p_refresh_mode) is called by RefreshMLTables::TransactionsML, the third table-building step of the daily ML chain (after MenuEntitiesML/ModifierEntitiesML, before UpsellAnalysisML).

Depending on p_refresh_mode, it DELETEs/TRUNCATEs and rebuilds this table from fact.transactionheader joined to fact.transactionitem, ml.weather, dim.itemcategory, dim.menuitem, dim.ordertype and dim.organizationlocation, aggregating to the (location, year, week) grain with businessdate/orderdatelocal-based windowing.

Three indexes — composite (locationid, yyyy, ww), businessdate, and composite (organizationid, locationid, menuitemid) — support both the weekly rollup pattern and item-level feature lookups.

6.7 ml.upsell_analysis

One row per (location, week, offer) upsell-funnel feature aggregate (20 columns) — built from fact.vw_offer_analysis for upsell-propensity model training.

DDL

```
CREATE TABLE ml.upsell_analysis (
    organizationid text,
    organizationname text,
    locationid text,
```

```
    locationname text,
    frequentcustomerid text,
    transactionheaderid text,
    recommendationid text,
    offereditem text,
    selecteditem text,
    item_class_type integer,
    upselltype text,
    quantity integer,
    businessdate date,
    orderdatelocal timestamp without time zone,
    yyyy integer,
    mm integer,
    dd integer,
    hh integer,
    ww integer,
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_ua_businessdate	btree	businessdate
ix_ml_ua_locid_yyyy_ww	btree	locationid, yyyy, ww
ix_ml_ua_yyyy_ww	btree	yyyy, ww

How It's Populated

ml.usp_refresh_upsell_analysis(p_refresh_mode) is called by RefreshMLTables::UpsellAnalysisML, run after TransactionsML in the daily ML chain.

Depending on p_refresh_mode, it DELETEs/TRUNCATEs and rebuilds this table from fact.transactionheader and fact.vw_offer_analysis joined to dim.organizationlocation and dim.menuitem, aggregating to the (location, year, week) grain.

Three indexes — businessdate, composite (locationid, yyyy, ww) and (yyyy, ww) alone — support both location-scoped and cross-location time-windowed queries.

6.8 ml.weather

One row per (location, date, hour) weather feature record (70 columns — the widest table in the schema, one column per weather attribute unpacked from dim.weather's JSONB payload) — joined into ml.transactions for weather-aware demand modeling.

DDL

```
CREATE TABLE ml.weather (
    organizationid text,
    organizationname text,
```

```
    locationid text,
    locationname text,
    weatherdate date,
    yyyy integer,
    mm integer,
    dd integer,
    ww integer,
    hh integer,
    humidity integer,
    condition text,
    temperature_c numeric(8,2),
    is_hot boolean,
    is_calm boolean,
    is_cold boolean,
    is_cool boolean,
    is_mild boolean,
    is_warm boolean,
    rain_mm numeric(8,2),
    is_sunny boolean,
    is_windy boolean,
    is_cloudy boolean,
    is_daytime boolean,
    is_raining boolean,
    is_snowing boolean,
    is_very_hot boolean,
    is_freezing boolean,
    is_overcast boolean,
    snowfall_mm numeric(8,2),
    temp_bucket text,
    wind_bucket text,
    feels_colder boolean,
    feels_hotter boolean,
    food_weather text,
    is_heavy_rain boolean,
    is_light_rain boolean,
    is_nighttime boolean,
    is_very_windy boolean,
    pressure_hpa numeric(8,2),
    weather_code integer,
    wind_gust_kmh numeric(8,2),
    comfort_score integer,
    drink_weather text,
    wind_speed_kmh numeric(8,2),
    comfort_bucket text,
    humidity_bucket text,
    condition_bucket text,
    is_precipitating boolean,
    precipitation_mm numeric(8,2),
    visibility_meters numeric(8,2),
    cloud_cover_percent numeric(8,2),
    is_unseasonably_hot boolean,
    is_unseasonably_cold boolean,
    outdoor_dining_score integer,
    wind_direction_degrees integer,
    precipitation_probability numeric(8,2),
    apparent_temperature_celsius numeric(8,2),
    sysinserttime timestamp without time zone
);
```

Constraints & Relationships

No primary key, unique, or foreign key constraint is defined for this table in the source DDL.

Indexes

Index Name	Method	Column(s)
ix_ml_wth_locationid_weatherdate_hh	btree	locationid, weatherdate, hh
ix_ml_wth_locid_yyyy_ww	btree	locationid, yyyy, ww
ix_ml_wth_weatherdate	btree	weatherdate
ix_ml_wth_yyyy_ww	btree	yyyy, ww

How It's Populated

ml.usp_refresh_weather(p_refresh_mode) is called by RefreshMLTables::WeatherML, the very first step of the daily ML refresh chain.

Depending on p_refresh_mode, it DELETEs/TRUNCATEs and rebuilds this table from dim.vw_weatherhourlydata (the view that unpacks dim.weather's JSONB payload into typed hourly columns) joined to dim.organizationlocation.

Because dim.weather itself is only populated by the DimensionWeatherInfo pipeline — which is not one of the four scheduled master triggers — this table's freshness is only as good as that separate, seemingly manually-run pipeline's last execution; worth confirming its schedule if weather-based ML features look stale.

Four indexes — composite (locationid, weatherdate, hh), composite (locationid, yyyy, ww), weatherdate alone, and (yyyy, ww) alone — support the several different time-grain query patterns used across the ML feature set.